Contents

1 Misc **1**
1.1 Templete . . . 1
1.2 Random Primes . . . 1
1.3 Pragmas . . . 1
1.4 Fast IO . . . 1
1.5 Day of Date . . . 2
1.6 Number of Days since 1-1-1 . . . 2
1.7 Int to Roman . . . 2
1.8 RNG . . . 2

2 Data Structures **2**
2.1 2D Segment Tree . . . 2
2.2 Fenwick RU-RQ . . . 3
2.3 Persistent Segment Tree . . . 3
2.4 Segment Tree Beats . . . 3
2.5 Li-Chao Tree . . . 4
2.6 Heavy-Light Decomposition . . . 4
2.7 Treap . . . 5
2.8 Unordered Map Custom Hash . . . 5
2.9 Interval Container . . . 5
2.10 Mo's on Tree . . . 5
2.11 DSU Rollback . . . 5
2.12 Link-Cut Tree . . . 5
2.13 Max Queue . . . 6
2.14 BigInt . . . 6
2.15 Xor Basis . . . 7
2.16 Wavelet Tree . . . 8

3 Dynamic Programming **8**
3.1 DP Convex Hull . . . 8
3.2 DP DNC . . . 8
3.3 DP Knuth-Yao . . . 8

4 Geometry **9**
4.1 Geometry Template . . . 9
4.2 Convex Hull . . . 10
4.3 Closest Pair of Points . . . 10
4.4 Smallest Enclosing Circle . . . 11
4.5 Sutherland-Hodgman Algorithm . . . 11
4.6 Centroid of Polygon . . . 12
4.7 Pick Theorem . . . 12

5 String **12**
5.1 KMP . . . 12
5.2 Aho-Corasick . . . 12
5.3 Manacher's Algorithm . . . 12
5.4 Z Algorithm . . . 12
5.5 Hashing . . . 13

6 Math **13**
6.1 Extended Euclidean GCD . . . 13
6.2 Generalized CRT . . . 13
6.3 Generalized Lucas Theorem . . . 13
6.4 Linear Diophantine . . . 14
6.5 Modular Linear Equation . . . 14
6.6 Miller-Rabin and Pollard's Rho . . . 14
6.7 Berlekamp-Massey . . . 14
6.8 Fast Fourier Transform . . . 15
6.9 Number Theoretic Transform . . . 15
6.10 Gauss-Jordan . . . 16
6.11 Josephus Problem . . . 16
6.12 Derangement . . . 16
6.13 Bernoulli Number . . . 16
6.14 Forbenius Number . . . 16
6.15 Stars and Bars with Upper Bound . . . 16
6.16 Arithmetic Sequences . . . 16
6.17 Mobius Function . . . 16
6.18 Floor Sum . . . 17
6.19 Combin Identity . . . 17
6.20 Fibonacci Identity . . . 17
6.21 Other Identity . . . 17
6.22 Lagrange Interpolation . . . 18
6.23 Discrete Root and Logarithm . . . 18
6.24 Matrix Expo . . . 19
6.25 Reflection Principle . . . 19

7 Graph **19**
7.1 Articulation Point and Bridge . . . 19
7.2 SCC and Strong Orientation . . . 19
7.3 Centroid Decomposition . . . 19
7.4 Dinic's Maximum Flow . . . 20
7.5 Minimum Cost Maximum Flow . . . 20
7.6 Flows with Demands . . . 20
7.7 Hungarian . . . 21
7.8 Edmond's Blossom . . . 21
7.9 Eulerian Path or Cycle . . . 21
7.10 Hierholzer's Algorithm . . . 22
7.11 2-SAT . . . 22
7.12 Edge Coloring . . . 22
7.13 K-th Shortest Path . . . 22

8 OEIS **22**
8.1 A000108 (Catalan Number) . . . 22
8.2 A000110 (Bell Numbers) . . . 23
8.3 A000127 . . . 23
8.4 A001434 . . . 23
8.5 A018819 . . . 23
8.6 A092098 . . . 23

1 Misc

1.1 Templete

```cpp
// Algoritma/Fungsi: Template dasar competitive programming dengan Fast I/O, 64-bit RNG, pb_ds ordered_set, dan gp_hash_table anti-hack.
// Kompleksitas Waktu: Fast I/O O(1), gp_hash_table average O(1) lookup, ordered_set O(log N) insert/query/order.
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// 64-bit RNG
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

// Custom hash to prevent anti-hash test cases
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

// pb_ds ordered_set & gp_hash_table
template<class T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template<class K, class V> using gpmap = gp_hash_table<K, V, custom_hash>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    return 0;
}
```

1.2 Random Primes

```
313, 100003, 200003, 1000003, 10000019, 15000017,
2147483647, 1000000087, 1878234273849638459,
2878234273849638443, 8878234273849638443,
9196969696969696969, 9206969696969696999,
998244353

Prime gap:
1e6: 114, 1e9: 282,
1e12: 540, 1e18: 1442
```

1.3 Pragmas

```cpp
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,tune=native")
#pragma GCC target("fpmath=387")
```

1.4 Fast IO

```cpp
// Algoritma/Fungsi: Fast I/O membaca dan menulis integer dan string menggunakan low-level I/O tak terkunci (unlocked I/O).
// Kompleksitas Waktu: O(jumlah digit) per operasi baca/tulis.
// NOTE: read() hanya membaca integer. Pastikan input tidak mengandung EOF tak terduga.
```


Page 1 of 23

```
//       write(long long) menangani LLONG_MIN secara khusus (lihat kode).
#include <bits/stdc++.h>
using namespace std;

#ifdef _WIN32
#define getchar_unlocked _getchar_nolock
#define putchar_unlocked _putchar_nolock
#endif

int read() {
    char c;
    do {
        c = getchar_unlocked();
    } while(c <= 32 && c != -1);
    int res = 0, mul = 1;
    if(c == '-') {
        mul = -1;
        c = getchar_unlocked();
    }
    while('0' <= c && c <= '9') {
        res = res * 10 + (c - '0');
        c = getchar_unlocked();
    }
    return res * mul;
}

void write(long long x) {
    if(x == 0) {
        putchar_unlocked('0');
        return;
    }
    if(x < 0) {
        putchar_unlocked('-');
        if(x == LLONG_MIN) {
            // Tangani khusus: -x dari LLONG_MIN overflow, tulis digit manual
            const char* s = "9223372036854775808";
            while(*s) putchar_unlocked(*s++);
            return;
        }
        x = -x;
    }
    char wbuf[25];
    int idx = 0;
    while(x > 0) {
        wbuf[idx++] = (x % 10) + '0';
        x /= 10;
    }
    for(int i = idx - 1; i >= 0; --i)
        putchar_unlocked(wbuf[i]);
}

void write(const char* s) {
    while(*s) {
        putchar_unlocked(*s);
        ++s;
    }
}
```

1.5 Day of Date

```
// 0-based: 0 = Sunday, 1 = Monday, ..., 6 = Saturday
const vector<int> T = {0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
int day(int d, int m, int y) {
    y -= (m < 3);
    return (y + y / 4 - y / 100 + y / 400 + T[m - 1] + d) % 7;
}
```

1.6 Number of Days since 1-1-1

```
// Algoritma Rata Die untuk menghitung total jumlah hari yang telah berlalu sejak 1 Januari 0001.
int rdn(int d, int m, int y) {
    if(m < 3) --y, m += 12;
    return 365 * y + y / 4 - y / 100 + y / 400 + (153 * m - 457) / 5 + d - 306;
}
```

1.7 Int to Roman

```
const string R[] = {
    "M", "CM", "D", "CD", "C", "XC", "L",
    "XL", "X", "IX", "V", "IV", "I"
};

const int N[] = {
    1000, 900, 500, 400, 100, 90, 50, 40, 10,
    9, 5, 4, 1
};

string to_roman(int x) {
    if(x == 0)
        return "N";
    string res = "";
    for(int i = 0; i < 13; ++i) {
        while(x >= N[i]) {
            x -= N[i];
            res += R[i];
        }
    }
    return res;
}
```

1.8 RNG

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template<class T>
T rand_int(T mn, T mx) { //inclusive
    return uniform_int_distribution<T>(mn, mx)(rng);
}
```

2 Data Structures

2.1 2D Segment Tree

```
// 2D Dynamic Segment Tree untuk query nilai minimum 2D dan update titik.
// Update O(log X log Y), Query O(log X log Y).

const int INF = 1e9;
const int MAX = 1e9;

struct Segtree2D {
  struct Segtree {
    struct node {
      int l, r, val;
      node* lc, * rc;
      node(int _l, int _r, int _val = INF) : l(_l), r(_r), val(_val),
        lc(NULL), rc(NULL) {}
    };
    typedef node* pnode;

    pnode root;

    Segtree(int l, int r) {
      root = new node(l, r);
    }

    void update(pnode& nw, int x, int val) {
      int l = nw->l, r = nw->r, mid = (l + r) / 2;
      if(l == r)
        nw->val = val;
      else {
        assert(l <= x && x <= r);
        pnode& child = x <= mid ? nw->lc : nw->rc;
        if(!child)
          child = new node(x, x, val);
        else if(child->l <= x && x <= child->r)
          update(child, x, val);
        else {
          do {
            if(x <= mid)
              r = mid;
            else
              l = mid + 1;
            mid = (l + r) / 2;
          } while((x <= mid) == (child->l <= mid));
          pnode nxt = new node(l, r);
          if(child->l <= mid)
            nxt->lc = child;
          else
            nxt->rc = child;
          child = nxt;
          update(nxt, x, val);
        }
        nw->val = min(nw->lc ? nw->lc->val : INF,
                      nw->rc ? nw->rc->val : INF);
      }
    }

    int query(pnode& nw, int x1, int x2) {
      if(!nw)
        return INF;
      int& l = nw->l, &r = nw->r;
      if(r < x1 || x2 < l)
        return INF;
      if(x1 <= l && r <= x2)
        return nw->val;
      int ret = min(query(nw->lc, x1, x2),
                    query(nw->rc, x1, x2));
      return ret;
    }

    void update(int x, int val) {
      assert(root->l <= x && x <= root->r);
      update(root, x, val);
    }

    int query(int l, int r) {
      return query(root, l, r);
    }
  };

  struct node {
    int l, r;
    Segtree y;
    node* lc, * rc;
    node(int _l, int _r) : l(_l), r(_r), y(0, MAX),
      lc(NULL), rc(NULL) {}
  };
```


New CP Leak
BINUS University

```
  typedef node* pnode;

  pnode root;

  Segtree2D(int l, int r) {
    root = new node(l, r);
  }

  void update(pnode& nw, int x, int y, int val) {
    int& l = nw->l, &r = nw->r, mid = (l + r) / 2;
    if(l == r)
      nw->y.update(y, val);
    else {
      if(x <= mid) {
        if(!nw->lc)
          nw->lc = new node(l, mid);
        update(nw->lc, x, y, val);
      } else {
        if(!nw->rc)
          nw->rc = new node(mid + 1, r);
        update(nw->rc, x, y, val);
      }
      val = min(nw->lc ? nw->lc->y.query(y, y) : INF,
                nw->rc ? nw->rc->y.query(y, y) : INF);
      nw->y.update(y, val);
    }
  }

  int query(pnode& nw, int x1, int x2, int y1, int y2) ↩
      {
    if(!nw)
      return INF;
    int& l = nw->l, &r = nw->r;
    if(r < x1 || x2 < l)
      return INF;
    if(x1 <= l && r <= x2)
      return nw->y.query(y1, y2);
    int ret = min(query(nw->lc, x1, x2, y1, y2),
                  query(nw->rc, x1, x2, y1, y2));
    return ret;
  }

  void update(int x, int y, int val) {
    assert(root->l <= x && x <= root->r);
    update(root, x, y, val);
  }

  int query(int x1, int x2, int y1, int y2) {
    return query(root, x1, x2, y1, y2);
  }
};
```

2.2 Fenwick RU-RQ

```
// 1-based index untuk range update dan range sum query↩
    .
// Range Update O(log N), Range Query O(log N).
using ll = long long;

struct FenwickRURQ {
    int n;
    vector<ll> bit1, bit2;
    FenwickRURQ(int _n) : n(_n), bit1(_n + 2, 0), bit2(↩
        _n + 2, 0) {}

    void updt(vector<ll>& bit, int idx, ll val) {
        for(; idx <= n; idx += idx & -idx)
            bit[idx] += val;
    }

    void update_range(int l, int r, ll val) {
        updt(bit1, l, val);
        updt(bit1, r + 1, -val);
        updt(bit2, l, val * (l - 1));
        updt(bit2, r + 1, -val * r);
    }

    ll query_prefix(int k) {
        ll s1 = 0, s2 = 0;
        for(int i = k; i > 0; i -= i & -i) {
            s1 += bit1[i];
            s2 += bit2[i];
        }
        return s1 * k - s2;
    }

    ll query_range(int l, int r) {
        return query_prefix(r) - query_prefix(l - 1);
    }
};
```

2.3 Persistent Segment Tree

```
// Persistent Segment Tree dengan node-copying untuk ↩
    query range sum pada versi histori masa lalu.
// Build O(N), Update O(log N), Query O(log N), Memori ↩
    O(N + Q log N).
#include <bits/stdc++.h>
using namespace std;

class PersistentSegtree{
private:
  int n, ptr, sz;
  struct P {
    int val = 0, l, r;
  };
  vector<P> node;
  vector<int> root;

  int newNode(){
    node.emplace_back();
    return ptr++;
  }
  int copyNode(int idx){
    node.push_back(node[idx]);
    return ptr++;
  }
  int build(int l, int r){
    int idx = newNode();
    if (l == r)
      return idx;
    node[idx].l = build(l, (l+r)/2);
    node[idx].r = build((l+r)/2 + 1, r);
    return idx;
  }
  int update(int idx, int l, int r, int x, int val){
    idx = copyNode(idx) ;
    if (l == r){
      node[idx].val += val;
      return idx;
    }
    int mid = (l+r)/2;
    if(x <= mid)
      node[idx].l = update(node[idx].l, l, mid, x, val)↩
          ;
    else
      node[idx].r = update(node[idx].r, mid+1, r, x, ↩
          val);
    node[idx].val = node[node[idx].l].val + node[node[↩
        idx].r].val;
    return idx;
  }
  int query(int idxl, int idxr, int l, int r, int x, ↩
      int y){
    if(y < l || r < x)
      return 0;
    if(x <= l && r <= y)
      return node[idxr].val - node[idxl].val;
    int mid = (l+r)/2;
    return query(node[idxl].l, node[idxr].l, l, mid, x,↩
        y)
      + query(node[idxl].r, node[idxr].r, mid+1, r, x, ↩
          y);
  }

public:
  PersistentSegtree(int _n) : n(_n), ptr(0){
    sz = 30*n;
    node.reserve(sz);
    root.push_back(build(1,n));
  }
  void update(int x, int val){
    root.push_back(update(root.back(), 1, n, x, val));
  }
  int query(int l, int r, int x, int y){
    return query(root[l-1], root[r], 1, n, x, y);
  }
};
```

2.4 Segment Tree Beats

```
// Segment Tree Beats untuk operasi range chmin (A_i = ↩
    min(A_i, v)), range max query, dan range sum query.
// Amortized O((N + Q) log N) untuk seluruh operasi.
using ll = long long;

const int N = 200005;
const ll INF = 1e18;

ll ST1[4 * N + 5], ST2[4 * N + 5];
ll smax[4 * N + 5], occ[4 * N + 5];
ll arr[N + 5];

void pushup(int idx) {
    ST1[idx] = max(ST1[2 * idx], ST1[2 * idx + 1]);
    ST2[idx] = ST2[2 * idx] + ST2[2 * idx + 1];
    if(ST1[2 * idx] == ST1[2 * idx + 1]) {
        smax[idx] = max(smax[2 * idx], smax[2 * idx + ↩
            1]);
        occ[idx] = occ[2 * idx] + occ[2 * idx + 1];
    } else {
        smax[idx] = min(ST1[2 * idx], ST1[2 * idx + 1])↩
            ;
        smax[idx] = max(smax[idx], max(smax[2 * idx], ↩
            smax[2 * idx + 1]));
        if(ST1[2 * idx] > ST1[2 * idx + 1])
            occ[idx] = occ[2 * idx];
        else
            occ[idx] = occ[2 * idx + 1];
    }
}

void build(int idx, int l, int r) {
    if(l == r) {
        ST1[idx] = ST2[idx] = arr[l];
        occ[idx] = 1;
        smax[idx] = -INF;
        return;
    }
    int mid = (l + r) >> 1;
```


New CP Leak
BINUS University
Page 3 of 23

```
    build(2 * idx, l, mid);
    build(2 * idx + 1, mid + 1, r);
    pushup(idx);
}

void pushdown(int idx, ll v) {
    if(ST1[idx] <= v) return;
    assert(v > smax[idx]);
    ST2[idx] -= (ST1[idx] - v) * occ[idx];
    ST1[idx] = v;
}

void down(int idx) {
    pushdown(2 * idx, ST1[idx]);
    pushdown(2 * idx + 1, ST1[idx]);
}

void update_chmin(int idx, int l, int r, int le, int ri, ll v) {
    if(r < le || ri < l || ST1[idx] <= v)
        return;
    if(le <= l && r <= ri && smax[idx] < v) {
        pushdown(idx, v);
        return;
    }
    down(idx);
    int mid = (l + r) >> 1;
    update_chmin(2 * idx, l, mid, le, ri, v);
    update_chmin(2 * idx + 1, mid + 1, r, le, ri, v);
    pushup(idx);
}

ll query_max(int idx, int l, int r, int le, int ri) {
    if(r < le || ri < l)
        return -INF;
    if(le <= l && r <= ri)
        return ST1[idx];
    down(idx);
    int mid = (l + r) >> 1;
    return max(query_max(2 * idx, l, mid, le, ri),
               query_max(2 * idx + 1, mid + 1, r, le, ri));
}

ll query_sum(int idx, int l, int r, int le, int ri) {
    if(r < le || ri < l)
        return 0;
    if(le <= l && r <= ri)
        return ST2[idx];
    down(idx);
    int mid = (l + r) >> 1;
    return query_sum(2 * idx, l, mid, le, ri) +
           query_sum(2 * idx + 1, mid + 1, r, le, ri);
}
```

2.5 Li-Chao Tree

```
// Dynamic Li-Chao Segment Tree untuk query nilai maksimum fungsi linear y = mx + c pada titik x.
// Insert Garis O(log C), Query Titik O(log C) dengan C adalah rentang koordinat.
using ll = long long;

struct Func {
  ll m, c;
  ll operator()(ll x) {
    return x * m + c;
  }
};

const int MAX = 1e9 + 1;
const ll INF = 1e18;
const Func NIL = {0, -INF};

struct Node {
  Func f;
  Node* lc;
  Node* rc;

  Node() : f(NIL), lc(nullptr), rc(nullptr) {}
  Node(const Node& n) : f(n.f), lc(nullptr), rc(nullptr) {}
};

Node* root = new Node;

void insert(Func f, Node* cur = root, int l = 0, int r = MAX - 1) {
  int m = l + (r - l) / 2;
  bool left = f(l) > cur->f(l);
  bool mid = f(m) > cur->f(m);
  if(mid)
    swap(f, cur->f);
  if(l != r) {
    if(left != mid) {
      if(!cur->lc)
        cur->lc = new Node(*cur);
      insert(f, cur->lc, l, m);
    } else {
      if(!cur->rc)
        cur->rc = new Node(*cur);
      insert(f, cur->rc, m + 1, r);
    }
  }
}

ll query(ll x, Node* cur = root, int l = 0, int r = MAX - 1) {
  if(!cur)
    return -INF;
  if(l == r)
    return cur->f(x);
  int m = l + (r - l) / 2;
  if(x <= m)
    return max(cur->f(x), query(x, cur->lc, l, m));
  else
    return max(cur->f(x), query(x, cur->rc, m + 1, r));
}
```

2.6 Heavy-Light Decomposition

```
// HLD pada Tree 1-based index untuk path update, path query, dan subtree update/query.
// Path Query/Update O(log^2 N), Subtree Query/Update O(log N).
struct Segtree {
    int n;
    vector<int> t;
    void init(int sz) { n = sz; t.assign(4 * n, 0); }
    void update(int l, int r, int x) { /* custom update */ }
    int query(int l, int r) { return 0; /* custom query */ }
};

class HLD {
private:
    vector<vector<int>> adj;
    vector<int> sz, in, out, nxt, par, depth;
    Segtree DS;
    int timer = -1;

    void dfs_sz(int u, int p = 0, int d = 0) {
        sz[u] = 1;
        par[u] = p;
        depth[u] = d;
        for(auto &v : adj[u]) {
            if(v != p) {
                dfs_sz(v, u, d + 1);
                sz[u] += sz[v];
            }
        }
        for(auto &v : adj[u]) {
            if(v != p && (adj[u][0] == p || sz[v] > sz[adj[u][0]])) {
                swap(v, adj[u][0]);
            }
        }
    }

    void dfs_hld(int u, int p = 0) {
        in[u] = ++timer;
        for(auto &v : adj[u]) {
            if(v != p) {
                nxt[v] = (v == adj[u][0] ? nxt[u] : v);
                dfs_hld(v, u);
            }
        }
        out[u] = timer;
    }

public:
    HLD(int n) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        in.resize(n + 1);
        out.resize(n + 1);
        nxt.assign(n + 1, 0);
        par.assign(n + 1, 0);
        depth.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void init(int root = 1) {
        dfs_sz(root, 0, 0);
        nxt[root] = root;
        dfs_hld(root, 0);
        DS.init(timer + 1);
    }

    void update_path(int u, int v, int x) {
        for(; nxt[u] != nxt[v]; v = par[nxt[v]]) {
            if(depth[nxt[u]] > depth[nxt[v]]) swap(u, v);
            DS.update(in[nxt[v]], in[v], x);
        }
        if(depth[u] > depth[v]) swap(u, v);
        DS.update(in[u], in[v], x);
    }

    int query_path(int u, int v) {
        int ans = 0;
        for(; nxt[u] != nxt[v]; v = par[nxt[v]]) {
            if(depth[nxt[u]] > depth[nxt[v]]) swap(u, v);
            ans = max(ans, DS.query(in[nxt[v]], in[v]));
```


New CP Leak
BINUS University
Page 4 of 23

```
        }
        if(depth[u] > depth[v]) swap(u, v);
        ans = max(ans, DS.query(in[u], in[v]));
        return ans;
    }

    void update_subtree(int u, int x) {
        DS.update(in[u], out[u], x);
    }

    int query_subtree(int u) {
        return DS.query(in[u], out[u]);
    }
};
```

2.7 Treap

```
// Implicit Treap (Cartesian Tree) untuk manipulasi sequence dengan operasi split by size, merge, insert, dan delete.
// Split dan Merge O(log N) expected.
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

using Key = int;

struct Treap {
  Key val;
  Treap* left;
  Treap* right;
  int prio, sz;
  Treap() {}
  Treap(int _val);
};

int size(Treap* tr) {
  return tr ? tr->sz : 0;
}

void update(Treap* tr) {
  tr->sz = 1 + size(tr->left) + size(tr->right);
}

Treap::Treap(Key _val) :
  val(_val), left(nullptr), right(nullptr), prio(rng()) {
  update(this);
}

pair<Treap*, Treap*> split(Treap* tr, int sz) {
  if(!tr) return {nullptr, nullptr};
  int left_sz = size(tr->left);
  if(sz <= left_sz) {
    auto [left, mid] = split(tr->left, sz);
    tr->left = mid;
    update(tr);
    return {left, tr};
  } else {
    auto [mid, right] = split(tr->right, sz - left_sz - 1);
    tr->right = mid;
    update(tr);
    return {tr, right};
  }
}

Treap* merge(Treap* l, Treap* r) {
  if(!l)
    return r;
  if(!r)
    return l;
  if(l->prio < r->prio) {
    l->right = merge(l->right, r);
    update(l);
    return l;
  } else {
    r->left = merge(l, r->left);
    update(r);
    return r;
  }
}
```

2.8 Unordered Map Custom Hash

```
// Custom Hash SplitMix64 untuk unordered_map dan unordered_set agar kebal terhadap collision test-case hacks.
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
      chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};

unordered_map<int, int, custom_hash> umap;
```

2.9 Interval Container

```
// Interval Container menggunakan set untuk memelihara himpunan interval disjoint (penambahan dan pengurangan interval).
// Amortized O(log N) per operasi penambahan/penghapusan interval.
// NOTE: Interval disimpan sebagai pair [L, R).
//       removeInterval meng-erase lalu re-insert, bukan cast-away constness (UB).
typedef pair<int, int> pii;

set<pii>::iterator addInterval(set<pii> &is, int L, int R){
  if (L == R) return is.end();
  auto it = is.lower_bound({L,R}), before = it;
  while (it != is.end() && it->first <= R){
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L){
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L, R});
}

void removeInterval(set<pii> &is, int L, int R){
  if(L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  int oldL = it->first;
  is.erase(it);
  if (oldL != L) is.emplace(oldL, L);
  if (R != r2) is.emplace(R, r2);
}
```

2.10 Mo's on Tree

Penjelasan formula Mo's Algorithm pada Tree menggunakan Flattening Euler Tour (in/out timer). $O((N+Q)*\sqrt{N}*T_{add})$.
$ST(u) \leq ST(v)$
$P = LCA(u, v)$
If $P = u$, query $[ST(u), ST(v)]$
Else query $[EN(u), ST(v)] + [ST(P), ST(P)]$

2.11 DSU Rollback

```
// DSU dengan riwayat perubahan untuk mendukung operasi rollback/undo.
// Find/Join O(log N), Rollback O(1) per operasi yang di-undo.
#define fi first
#define se second
typedef vector<int> vi;
typedef pair<int, int> pii;

struct RollbackUF{
  vi e;
  vector<pii> st;
  RollbackUF(int n) : e(n, -1){}
  int size (int x){
    return -e[find(x)];
  }
  int find(int x){
    return (e[x] < 0) ? x : find(e[x]);
  }
  int time(){
    return st.size();
  }
  void rollback(int t){
    for (int i = time(); i-->t; )
      e[st[i].fi] = st[i].se;
    st.resize(t);
  }
  bool join(int a, int b){
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b];
    e[b] = a;
    return true;
  }
};
```

2.12 Link-Cut Tree

```
// Link-Cut Tree (LCT) berbasis Splay Tree untuk konektivitas dinamis graf pohon, query path, dan pemeliharaan subtree.
// Amortized O(log N) per operasi link, cut, makeRoot, dan path query.
typedef long long ll;
const int maxV = 100005;

typedef struct Node* Np;
struct Node {
  Np p, c[2];
  bool flip = 0;
```


New CP Leak
BINUS University
Page 5 of 23

```
int vtx;
ll val, sz;
int sub, vsub = 0;
ll stsum;

Node(int _vtx, int _val = 1) : vtx(_vtx), val(_val) {
  p = c[0] = c[1] = nullptr;
  calc();
}

friend int getSz(Np x) { return x ? x->sz : 0; }
friend int getSub(Np x) { return x ? x->sub : 0; }
friend ll getStSum(Np x) { return x ? x->stsum : 0; }

void prop() {
  if(!flip) return;
  swap(c[0], c[1]);
  flip = 0;
  for(int i = 0; i < 2; i++) {
    if(c[i]) c[i]->flip ^= 1;
  }
}

void calc() {
  for(int i = 0; i < 2; i++) {
    if(c[i]) c[i]->prop();
  }
  sz = 1 + getSz(c[0]) + getSz(c[1]);
  sub = 1 + getSub(c[0]) + getSub(c[1]) + vsub;
  stsum = val + getStSum(c[0]) + getStSum(c[1]);
}

int dir() {
  if(!p) return -2;
  for(int i = 0; i < 2; i++) {
    if(p->c[i] == this) return i;
  }
  return -1;
}

bool isRoot() { return dir() < 0; }

friend void setLink(Np x, Np y, int d) {
  if(y) y->p = x;
  if(d >= 0) x->c[d] = y;
}

void rot() {
  assert(!isRoot());
  int x = dir();
  Np pa = p;
  setLink(pa->p, this, pa->dir());
  setLink(pa, c[x ^ 1], x);
  setLink(this, pa, x ^ 1);
  pa->calc();
}

void splay() {
  while(!isRoot() && !p->isRoot()) {
    p->p->prop(), p->prop(), prop();
    dir() == p->dir() ? p->rot() : rot();
    rot();
  }
  if(!isRoot()) p->prop(), prop(), rot();
  prop();
  calc();
}

Np fbo(int b) {
  prop();
  int z = getSz(c[0]);
  if(b == z) {
    splay();
    return this;
  }
  return b < z ? c[0]->fbo(b) : c[1]->fbo(b - z - 1);
}

void access() {
  for(Np v = this, pre = nullptr; v; v = v->p) {
    v->splay();
    if(pre) v->vsub -= pre->sub;
    if(v->c[1]) v->vsub += v->c[1]->sub;
    v->c[1] = pre;
    v->calc();
    pre = v;
  }
  splay();
  assert(!c[1]);
}

void makeRoot() {
  access();
  flip ^= 1;
  access();
  assert(!c[0] && !c[1]);
}

friend Np lca(Np x, Np y) {
  if(x == y) return x;
  x->access(), y->access();
  if(!x->p) return nullptr;
  x->splay();
  return x->p ? x->p : x;
}

friend bool connected(Np x, Np y) { return lca(x, y) ↩
    != nullptr; }

int distRoot() {
  access();
  return getSz(c[0]);
}

Np getRoot() {
  access();
  Np a = this;
  while(a->c[0]) a = a->c[0], a->prop();
  a->access();
  return a;
}

Np getPar(int b) {
  access();
  b = getSz(c[0]) - b;
  assert(b >= 0);
  return fbo(b);
}

void setVal(int v) { access(); val = v; calc(); }
void addVal(int v) { access(); val += v; calc(); }

friend void link(Np x, Np y, bool force = 1) {
  assert(!connected(x, y));
  if(force) {
    y->makeRoot();
  } else {
    y->access();
    assert(!y->c[0]);
  }
  x->access();
  setLink(y, x, 0);
  y->calc();
}

friend void cut(Np y) {
  y->access();
  assert(y->c[0]);
  y->c[0]->p = NULL;
  y->c[0] = NULL;
  y->calc();
}

friend void cut(Np x, Np y) {
  x->makeRoot();
  y->access();
  assert(y->c[0] == x && !x->c[0] && !x->c[1]);
  cut(y);
}
};
Np LCT[maxV];
```

2.13 Max Queue

```
// Monotonic Sliding Window Queue untuk mencari nilai ↩
    maksimum dalam antrian bergerak secara efisien.
// Amortized O(1) per operasi add dan remove, O(1) ↩
    untuk getMax().
// NOTE: Pastikan queue tidak kosong sebelum memanggil ↩
    getMax().
//       Untuk MinQueue, ganti perbandingan < menjadi >↩
     pada fungsi add.
struct MaxQueue{
  deque<pair<int,int>> q;
  int cntAdd, cntRem;

  MaxQueue() : cntAdd(0), cntRem(0) {};

  void add(int x){
    while(!q.empty()&&q.back().first<x)q.pop_back();
    q.push_back({x,cntAdd++});
  }
  void remove(){
    if (!q.empty() && q.front().second == cntRem)
      q.pop_front();
    cntRem++;
  }
  int getMax(){
    assert(!q.empty() && "getMax() called on empty ↩
        MaxQueue");
    return q.front().first;
  }
};
```

2.14 BigInt

```
using ll = long long;

struct BigInt {
    vector<int> d; // Digit dibalik: d[0] satuan

    BigInt(ll v = 0) {
        do { d.push_back(v % 10); v /= 10; } while (v >↩
            0);
    }
    BigInt(string s) {
        for (int i = (int)s.size() - 1; i >= 0; i--)
            if (isdigit(s[i])) d.push_back(s[i] - '0');
        trim();
    }
    void trim() {
```


New CP Leak
BINUS University

Page 6 of 23

```
        while (d.size() > 1 && d.back() == 0) d.↩
            pop_back();
    }

    bool operator<(const BigInt &o) const {
        if (d.size() != o.d.size()) return d.size() < o↩
            .d.size();
        for (int i = (int)d.size() - 1; i >= 0; i--)
            if (d[i] != o.d[i]) return d[i] < o.d[i];
        return false;
    }
    bool operator==(const BigInt &o) const { return d ↩
        == o.d; }
    bool operator<=(const BigInt &o) const { return *↩
        this < o || *this == o; }
    bool operator>(const BigInt &o) const { return o < ↩
        *this; }
    bool operator>=(const BigInt &o) const { return !(*↩
        this < o); }
    bool operator!=(const BigInt &o) const { return !(*↩
        this == o); }

    BigInt operator+(const BigInt &o) const {
        BigInt res; res.d.clear();
        int c = 0;
        for (size_t i = 0; i < max(d.size(), o.d.size()↩
            ) || c; i++) {
            if (i < d.size()) c += d[i];
            if (i < o.d.size()) c += o.d[i];
            res.d.push_back(c % 10);
            c /= 10;
        }
        return res;
    }

    BigInt operator-(const BigInt &o) const { // Asumsi↩
        : *this >= o
        BigInt res = *this;
        int c = 0;
        for (size_t i = 0; i < res.d.size(); i++) {
            int sub = (i < o.d.size() ? o.d[i] : 0) + c↩
                ;
            if (res.d[i] < sub) { res.d[i] += 10 - sub;↩
                c = 1; }
            else { res.d[i] -= sub; c = 0; }
        }
        res.trim();
        return res;
    }

    BigInt operator*(const BigInt &o) const {
        if (*this == 0 || o == 0) return 0;
        BigInt res; res.d.assign(d.size() + o.d.size(),↩
            0);
        for (size_t i = 0; i < d.size(); i++) {
            for (size_t j = 0; j < o.d.size(); j++) {
                res.d[i + j] += d[i] * o.d[j];
                res.d[i + j + 1] += res.d[i + j] / 10;
                res.d[i + j] %= 10;
            }
        }
        res.trim();
        return res;
    }

    BigInt operator/(const BigInt &o) const {
        if (o == 0) throw runtime_error("Division by ↩
            zero");
        BigInt q, r = 0;
        q.d.resize(d.size());
        for (int i = (int)d.size() - 1; i >= 0; i--) {
            if (r == 0) r.d.clear();
            r.d.insert(r.d.begin(), d[i]);
            r.trim();
            int cnt = 0;
            while (!(r < o)) { r = r - o; cnt++; }
            q.d[i] = cnt;
        }
        q.trim();
        return q;
    }

    BigInt operator%(const BigInt &o) const {
        if (o == 0) throw runtime_error("Division by ↩
            zero");
        BigInt r = 0;
        for (int i = (int)d.size() - 1; i >= 0; i--) {
            if (r == 0) r.d.clear();
            r.d.insert(r.d.begin(), d[i]);
            r.trim();
            while (!(r < o)) r = r - o;
        }
        return r;
    }

    friend ostream& operator<<(ostream &out, const ↩
        BigInt &x) {
        for (int i = (int)x.d.size() - 1; i >= 0; i--) ↩
            out << x.d[i];
        return out;
    }
    friend istream& operator>>(istream &in, BigInt &x) ↩
        {
        string s; if (in >> s) x = BigInt(s); return in↩
            ;
    }
};

BigInt power(BigInt a, ll b) {
    BigInt res = 1;
    while (b) {
        if (b & 1) res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}

BigInt gcd(BigInt a, BigInt b) {
    while (!(b == 0)) {
        BigInt r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

2.15 Xor Basis

```
using ll = long long;

template<ll BITS = 60>
struct XorBasis {
    ll basis[BITS];
    ll sz = 0;

    XorBasis() {
        memset(basis, 0, sizeof(basis));
    }

    // Memasukkan elemen ke dalam basis.
    // Mengembalikan 1 (true) jika elemen independen, 0↩
        (false) jika tidak.
    bool insert(ll mask) {
        for (ll i = BITS - 1; i >= 0; --i) {
            if ((mask >> i) & 1) {
                if (!basis[i]) {
                    basis[i] = mask;
                    sz++;
                    return true;
                }
                mask ^= basis[i];
            }
        }
        return false;
    }

    // Memeriksa apakah suatu nilai bisa dibentuk dari ↩
        subset elemen
    bool can_form(ll mask) const {
        for (ll i = BITS - 1; i >= 0; --i) {
            if ((mask >> i) & 1) {
                if (!basis[i]) return false;
                mask ^= basis[i];
            }
        }
        return true;
    }

    // Mendapatkan nilai XOR maksimum yang bisa ↩
        dibentuk
    ll get_max() const {
        ll ans = 0;
        for (ll i = BITS - 1; i >= 0; --i) {
            if ((ans ^ basis[i]) > ans) {
                ans ^= basis[i];
            }
        }
        return ans;
    }

    // Mendapatkan nilai XOR minimum (non-zero) yang ↩
        bisa dibentuk
    ll get_min() const {
        for (ll i = 0; i < BITS; ++i) {
            if (basis[i]) return basis[i];
        }
        return 0;
    }

    // Menggabungkan dua basis
    void merge(const XorBasis& other) {
        for (ll i = BITS - 1; i >= 0; --i) {
            if (other.basis[i]) {
                insert(other.basis[i]);
            }
        }
    }

    // Mendapatkan nilai XOR terkecil ke-k (1-indexed) ↩
        dari semua kombinasi subset distinct
    ll kth_smallest(ll k) const {
        // Reduksi basis ke bentuk kanonik (Reduced Row↩
            Echelon Form)
        ll reduced[BITS];
        memcpy(reduced, basis, sizeof(basis));
        for (ll i = 0; i < BITS; ++i) {
            if (!reduced[i]) continue;
            for (ll j = i + 1; j < BITS; ++j) {
                if ((reduced[j] >> i) & 1) {
                    reduced[j] ^= reduced[i];
                }
```

```
            }
        }

        vector<ll> v;
        for (ll i = 0; i < BITS; ++i) {
            if (reduced[i]) v.push_back(reduced[i]);
        }

        ll num_distinct = (1LL << (ll)v.size());
        if (k > num_distinct) return -1; // k melebihi ↩
             jumlah kombinasi unik

        ll ans = 0;
        for (ll i = 0; i < (ll)v.size(); ++i) {
            if ((k >> i) & 1) {
                ans ^= v[i];
            }
        }
        return ans;
    }
};
```

2.16 Wavelet Tree

```
struct WaveletTree {
    int lo, hi;
    WaveletTree *left = nullptr, *right = nullptr;
    vector<int> pref;

    // Build tree dari iterator [from, to) dengan ↩
         domain nilai [lo, hi]
    WaveletTree(auto from, auto to, int lo, int hi) : ↩
         lo(lo), hi(hi) {
        if (lo == hi || from >= to) return;
        int mid = lo + (hi - lo) / 2;

        pref.reserve(distance(from, to) + 1);
        pref.push_back(0);
        for (auto it = from; it != to; ++it) {
            pref.push_back(pref.back() + (*it <= mid));
        }

        auto pivot = stable_partition(from, to, [mid](↩
             int x) {
            return x <= mid;
        });

        left = new WaveletTree(from, pivot, lo, mid);
        right = new WaveletTree(pivot, to, mid + 1, hi)↩
             ;
    }

    // 1. K-th Smallest di range [l, r] (1-indexed) -> ↩
         O(log Sigma)
    int kth(int l, int r, int k) {
        if (lo == hi) return lo;
        int in_left = pref[r] - pref[l - 1];
        if (k <= in_left) {
            return left->kth(pref[l - 1] + 1, pref[r], ↩
                 k);
        } else {
            return right->kth(l - pref[l - 1], r - pref↩
                 [r], k - in_left);
        }
    }

    // 2. Menghitung banyaknya elemen <= x di range [l,↩
          r] -> O(log Sigma)
    int count_lte(int l, int r, int x) {
        if (l > r || x < lo) return 0;
        if (hi <= x) return r - l + 1;
        int mid = lo + (hi - lo) / 2;
        int in_left = pref[r] - pref[l - 1];
        if (x <= mid) {
            return left ? left->count_lte(pref[l - 1] +↩
                 1, pref[r], x) : 0;
        } else {
            return in_left + (right ? right->count_lte(↩
                 l - pref[l - 1], r - pref[r], x) : 0);
        }
    }

    // 3. Menghitung frekuensi kemunculan nilai x di ↩
         range [l, r] -> O(log Sigma)
    int count_freq(int l, int r, int x) {
        if (l > r || x < lo || x > hi) return 0;
        if (lo == hi) return r - l + 1;
        int mid = lo + (hi - lo) / 2;
        if (x <= mid) {
            return left ? left->count_freq(pref[l - 1] ↩
                 + 1, pref[r], x) : 0;
        } else {
            return right ? right->count_freq(l - pref[l↩
                 - 1], r - pref[r], x) : 0;
        }
    }

    ~WaveletTree() {
        delete left;
        delete right;
    }
};
```

3 Dynamic Programming

3.1 DP Convex Hull

```
// Kondisi pengecekan gradien pada Convex Hull Trick (↩
     CHT) untuk eliminasi garis non-optimal pada ↩
     optimasi DP.
// Amortized O(1) per penambahan garis, keseluruhan DP ↩
     O(N).
// NOTE: Menggunakan cross-multiplication (long long) ↩
     agar tidak ada floating-point precision error.
//       Pastikan slope (m[]) monoton naik/turun agar ↩
     CHT bekerja dengan benar.
typedef long long ll;

const int N = 100005;
int y_coord[N], m[N];

// Returns true if line b is redundant between line a ↩
     and line c
// Menggunakan cross-multiplication: (yb-ya)*(mb-mc) <=↩
      (yc-yb)*(ma-mb)
bool cekx(int a, int b, int c) {
  return (ll)(y_coord[b] - y_coord[a]) * (m[b] - m[c]) ↩
       <= (ll)(y_coord[c] - y_coord[b]) * (m[a] - m[b]);
}
```

3.2 DP DNC

```
// Divide and Conquer (D&C) DP Optimization untuk ↩
     membagi N elemen ke dalam K partisi dengan fungsi ↩
     biaya monoton.
// O(K * N log N).
typedef long long ll;

const ll INF = 1e18;
const int N = 1005, K = 105;
ll dp[K][N];
ll c[N][N];

void compute(int rem, int l, int r, int optl, int optr)↩
      {
    if(l > r) return;
    int mid = (l + r) >> 1;
    ll opt_val = INF;
    int opt_id = -1;

    for(int i = optl; i <= min(mid - 1, optr); ++i) {
        ll cur = dp[rem - 1][i] + c[i][mid];
        if(cur < opt_val) {
            opt_val = cur;
            opt_id = i;
        }
    }

    dp[rem][mid] = opt_val;
    if(opt_id == -1) opt_id = optl;
    compute(rem, l, mid - 1, optl, opt_id);
    compute(rem, mid + 1, r, opt_id, optr);
}

void solve(int n, int k) {
    for(int i = 1; i <= n; ++i) dp[1][i] = c[0][i];
    for(int rem = 2; rem <= k; ++rem) {
        compute(rem, rem, n, rem - 1, n);
    }
}
```

3.3 DP Knuth-Yao

```
// Knuth-Yao DP Optimization untuk optimasi interval DP↩
     dengan kondisi quadrangle inequality (opt[i][j-1] ↩
     <= opt[i][j] <= opt[i+1][j]).
// O(N^2) (tereduksi dari naif O(N^3)).
// NOTE: cost[i][j] harus memenuhi quadrangle ↩
     inequality agar optimasi valid.
//       Split point k harus memenuhi i < k < j (strict↩
     ) agar tidak self-reference.
typedef long long ll;

const ll INF = 1e18;
const int N = 1005;
ll dp[N][N];
int opt[N][N];
ll cost[N][N];

void knuth_yao(int n) {
    for(int i = 0; i <= n; ++i) {
        for(int j = 0; j <= n; ++j) {
            dp[i][j] = (j <= i + 1 ? 0 : INF);
            opt[i][j] = i + 1;
        }
    }

    for(int len = 2; len <= n; len++) {
        for(int i = 0; i + len <= n; i++) {
            int j = i + len;
            int sta = opt[i][j - 1];
            int end = opt[i + 1][j];
            dp[i][j] = INF;
            for(int k = max(sta, i + 1); k <= min(end, ↩
                 j - 1); k++) {
                ll val = dp[i][k] + dp[k][j] + cost[i][↩
                     j];
                if(val < dp[i][j]) {
```


New CP Leak
BINUS University
Page 8 of 23

```
                    dp[i][j] = val;
                    opt[i][j] = k;
                }
            }
        }
    }
}
```

4 Geometry

4.1 Geometry Template

```
#define For(i, a, b) for(int i = (a); i <= (b); ++i)
#define rep(i, a, b) for(int i = (a); i <= (b); ++i)
#define pb push_back

const double PI = acos(-1.0);
const double INFD = 1E18;
const double EPS = 1e-9;

inline bool same_d(double x, double y) {
    return fabs(x - y) < EPS;
}

inline bool between_d(double x, double l, double r) {
    return (min(l, r) <= x + EPS && x <= max(l, r) + ↩
        EPS);
}

inline double dabs(double x) {
    return fabs(x);
}

struct point {
    double x, y;
    point() : x(0.0), y(0.0) {}
    point(double _x, double _y) : x(_x), y(_y) {}

    bool operator<(const point& other) const {
        if(fabs(x - other.x) > EPS) return x < other.x;
        return y < other.y - EPS;
    }
    bool operator==(const point& other) const {
        return same_d(x, other.x) && same_d(y, other.y)↩
            ;
    }
};

double e_dist(point P1, point P2) {
    return hypot(P1.x - P2.x, P1.y - P2.y);
}

double m_dist(point P1, point P2) {
    return dabs(P1.x - P2.x) + dabs(P1.y - P2.y);
}

bool pointBetween(point P, point L, point R) {
    return fabs(e_dist(L, P) + e_dist(P, R) - e_dist(L,↩
        R)) < EPS;
}

bool collinear(point P, point L, point R) {
    return dabs(P.x * (L.y - R.y) + L.x * (R.y - P.y) +↩
        R.x * (P.y - L.y)) < EPS;
}

point mid(point P, point Q) {
    return point((P.x + Q.x) / 2, (P.y + Q.y) / 2);
}

struct vec {
    double x, y;
    vec() : x(0.0), y(0.0) {}
    vec(double _x, double _y) : x(_x), y(_y) {}
    vec(point A) : x(A.x), y(A.y) {}
    vec(point A, point B) : x(B.x - A.x), y(B.y - A.y) ↩
        {}
};

vec scale(vec v, double s) { return vec(v.x * s, v.y * ↩
    s); }
vec flip(vec v) { return vec(-v.x, -v.y); }
double dot(vec u, vec v) { return (u.x * v.x + u.y * v.↩
    y); }
double cross(vec u, vec v) { return (u.x * v.y - u.y * ↩
    v.x); }
double norm_sq(vec v) { return (v.x * v.x + v.y * v.y);↩
     }

point translate(point P, vec v) { return point(P.x + v.↩
    x, P.y + v.y); }
point rotate(point P, point O, double angle) {
    vec v(O);
    P = translate(P, flip(v));
    return translate(point(P.x * cos(angle) - P.y * sin↩
        (angle),
                           P.x * sin(angle) + P.y * cos↩
                               (angle)), v);
}

double angle(point A, point O, point B) {
    vec OA(O, A), OB(O, B);
    return acos(dot(OA, OB) / sqrt(norm_sq(OA) * ↩
        norm_sq(OB)));
}

int orientation(point P, point Q, point R) {
    vec PQ(P, Q), PR(P, R);
    double c = cross(PQ, PR);
    if(c < -EPS) return -1;
    if(c > EPS) return 1;
    return 0;
}

struct line {
    double a, b, c;
    line() : a(0), b(0), c(0) {}
    line(double _a, double _b, double _c) : a(_a), b(_b↩
        ), c(_c) {}
    line(point P1, point P2) {
        if(same_d(P1.x, P2.x))
            a = 1.0, b = 0.0, c = -P1.x;
        else
            a = -(P1.y - P2.y) / (P1.x - P2.x),
            b = 1.0, c = -(a * P1.x) - P1.y;
    }
    line(point P, double slope) {
        if(same_d(slope, INFD))
            a = 1.0, b = 0.0, c = -P.x;
        else
            a = -slope, b = 1.0, c = -(a * P.x) - P.y;
    }
    bool operator==(line other) {
        return same_d(a, other.a) && same_d(b, other.b)↩
            && same_d(c, other.c);
    }
    double slope() {
        if(same_d(b, 0.0)) return INFD;
        return -(a / b);
    }
};

bool paralel(line L1, line L2) {
    return same_d(L1.a * L2.b, L1.b * L2.a);
}

bool intersection(line L1, line L2, point &P) {
    if(paralel(L1, L2)) return false;
    P.x = (L2.b * L1.c - L1.b * L2.c) / (L2.a * L1.b - ↩
        L1.a * L2.b);
    if(same_d(L1.b, 0.0))
        P.y = -(L2.a * P.x + L2.c) / L2.b;
    else
        P.y = -(L1.a * P.x + L1.c) / L1.b;
    return true;
}

double pointToLine(point P, point A, point B, point &C)↩
     {
    vec AP(A, P), AB(A, B);
    double u = dot(AP, AB) / norm_sq(AB);
    C = translate(A, scale(AB, u));
    return e_dist(P, C);
}

double lineToLine(line L1, line L2) {
    if(!paralel(L1, L2)) return 0.0;
    return dabs(L2.c - L1.c) / sqrt(L1.a * L1.a + L1.b ↩
        * L1.b);
}

struct segment {
    point P, Q;
    line L;
    segment() : P(point()), Q(point()), L(line()) {}
    segment(point _P, point _Q) : P(_P), Q(_Q), L(line(↩
        _P, _Q)) {}
    bool operator==(segment other) {
        return P == other.P && Q == other.Q;
    }
};

bool onSegment(point P, segment S) {
    if(orientation(S.P, S.Q, P) != 0) return false;
    return between_d(P.x, S.P.x, S.Q.x) && between_d(P.↩
        y, S.P.y, S.Q.y);
}

bool s_intersection(segment S1, segment S2) {
    double o1 = orientation(S1.P, S1.Q, S2.P);
    double o2 = orientation(S1.P, S1.Q, S2.Q);
    double o3 = orientation(S2.P, S2.Q, S1.P);
    double o4 = orientation(S2.P, S2.Q, S1.Q);
    if(o1 != o2 && o3 != o4) return true;
    if(o1 == 0 && onSegment(S2.P, S1)) return true;
    if(o2 == 0 && onSegment(S2.Q, S1)) return true;
    if(o3 == 0 && onSegment(S1.P, S2)) return true;
    if(o4 == 0 && onSegment(S1.Q, S2)) return true;
    return false;
}

double pointToSegment(point P, point A, point B, point ↩
    &C) {
    vec AP(A, P), AB(A, B);
    double u = dot(AP, AB) / norm_sq(AB);
    if(u < EPS) {
        C = A;
        return e_dist(P, A);
    }
    if(u + EPS > 1.0) {
        C = B;
```


New CP Leak
BINUS University
Page 9 of 23

```
        return e_dist(P, B);
    }
    return pointToLine(P, A, B, C);
}

double segmentToSegment(segment S1, segment S2) {
    if(s_intersection(S1, S2)) return 0.0;
    double ret = INFD;
    point dummy;
    ret = min({ret,
               pointToSegment(S1.P, S2.P, S2.Q, dummy),
               pointToSegment(S1.Q, S2.P, S2.Q, dummy),
               pointToSegment(S2.P, S1.P, S1.Q, dummy),
               pointToSegment(S2.Q, S1.P, S1.Q, dummy)
                   });
    return ret;
}

point lineIntersectSeg(point p, point q, point A, point
      B) {
    double a = B.y - A.y;
    double b = A.x - B.x;
    double c = B.x * A.y - A.x * B.y;
    double u = fabs(a * p.x + b * p.y + c);
    double v = fabs(a * q.x + b * q.y + c);
    return point((p.x * v + q.x * u) / (u + v),
                 (p.y * v + q.y * u) / (u + v));
}

struct circle {
    point P;
    double r;
    circle() : P(point()), r(0.0) {}
    circle(point _P, double _r) : P(_P), r(_r) {}
    circle(point P1, point P2) : P(mid(P1, P2)), r(
         e_dist(P, P1)) {}
    circle(point P1, point P2, point P3) {
        point M1 = mid(P1, P2), M2 = mid(P2, P3);
        point Q2 = rotate(P2, P1, PI / 2), Q3 = rotate(
             P3, P2, PI / 2);
        vec P1Q2(P1, Q2), P2Q3(P2, Q3);
        point M3 = translate(M1, P1Q2), M4 = translate(
             M2, P2Q3);
        line L1(M1, M3), L2(M2, M4);
        intersection(L1, L2, P);
        r = e_dist(P, P1);
    }
    bool operator==(circle other) {
        return P == other.P && same_d(r, other.r);
    }
};

bool insideCircle(point P, circle C) {
    return e_dist(P, C.P) <= C.r + EPS;
}

// NOTE: line menggunakan konvensi ax+by+c=0. Fungsi
     ini mensubstitusi ke persamaan lingkaran.
//       Jika b==0, garis vertikal ditangani terpisah.
bool lc_intersection(line L, circle O, point &P1, point
      &P2) {
    double a = L.a, b = L.b, c = L.c;
    double x = O.P.x, y = O.P.y, r = O.r;
    if(same_d(b, 0.0)) {
        double px = -c / a;
        if(fabs(x - px) > r + EPS) return false;
        double dy = sqrt(max(0.0, r * r - (px - x) * (
             px - x)));
        P1 = point(px, y - dy);
        P2 = point(px, y + dy);
        return true;
    }
    // Substitusi y = (-c - ax) / b ke (x-cx)^2 + (y-cy
         )^2 = r^2
    double A = a * a + b * b;
    double B = 2 * a * c + 2 * a * b * y - 2 * b * b *
         x;
    double C = b * b * x * x + c * c + 2 * b * c * y +
         b * b * y * y - b * b * r * r;
    double D = B * B - 4 * A * C;
    if(D < -EPS) return false;
    D = sqrt(max(0.0, D));
    P1.x = (-B - D) / (2 * A);
    P1.y = (-c - a * P1.x) / b;
    P2.x = (-B + D) / (2 * A);
    P2.y = (-c - a * P2.x) / b;
    return true;
}

struct polygon {
    vector<point> P;
    polygon() { P.clear(); }
    polygon(const vector<point>& _P) : P(_P) {}
};

bool rayCast(point P, polygon &A) {
    bool inside = false;
    int n = A.P.size();
    for(int i = 0, j = n - 1; i < n; j = i++) {
        if(((A.P[i].y > P.y) != (A.P[j].y > P.y)) &&
           (P.x < (A.P[j].x - A.P[i].x) * (P.y - A.P[i
                ].y) / (A.P[j].y - A.P[i].y) + A.P[i].x)
                )
            inside = !inside;
    }
    return inside;
}

double DP[110][110];
double minCostPolygonTriangulation(polygon &A) {
    if(A.P.size() < 3) return 0;
    For(i, 0, (int)A.P.size()) {
        for(int j = 0, k = i; k < (int)A.P.size(); j++,
             k++) {
            if(k < j + 2)
                DP[j][k] = 0.0;
            else {
                DP[j][k] = INFD;
                rep(l, j + 1, k - 1) {
                    double cost = e_dist(A.P[j], A.P[k
                         ]) + e_dist(A.P[k], A.P[l]) +
                         e_dist(A.P[l], A.P[j]);
                    DP[j][k] = min(DP[j][k], DP[j][l] +
                         DP[l][k] + cost);
                }
            }
        }
    }
    return DP[0][A.P.size() - 1];
}
```

4.2 Convex Hull

```
// Andrew's Monotone Chain untuk mencari Convex Hull 2D
     dan Winding Number untuk point-in-polygon test.
// Convex Hull O(N log N), Point in Polygon O(N).
typedef long double TD;
namespace GEOM {
    typedef pair<TD, TD> Pt;
    const TD EPS = 1e-9;
    TD cross(Pt o, Pt a, Pt b) {
        return (a.first - o.first) * (b.second - o.
             second) -
               (a.second - o.second) * (b.first - o.
                    first);
    }

    TD cross(Pt a, Pt b) {
        return a.first * b.second - a.second * b.first;
    }

    TD dist(Pt a, Pt b) {
        return hypot(a.first - b.first, a.second - b.
             second);
    }

    TD shoelaceX2(const vector<Pt>& convHull) {
        TD ret = 0;
        int n = convHull.size();
        for(int i = 0; i < n; i++)
            ret += cross(convHull[i], convHull[(i + 1)
                 % n]);
        return fabs(ret);
    }

    vector<Pt> createConvexHull(vector<Pt> pts) {
        int n = pts.size(), k = 0;
        if(n <= 2) return pts;
        vector<Pt> h(2 * n);
        sort(pts.begin(), pts.end());

        for(int i = 0; i < n; ++i) {
            while(k >= 2 && cross(h[k - 2], h[k - 1],
                 pts[i]) <= 0)
                k--;
            h[k++] = pts[i];
        }

        for(int i = n - 2, t = k + 1; i >= 0; i--) {
            while(k >= t && cross(h[k - 2], h[k - 1],
                 pts[i]) <= 0)
                k--;
            h[k++] = pts[i];
        }
        h.resize(k - 1);
        return h;
    }

    bool isInside(Pt pv, const vector<Pt>& x) {
        int n = x.size(), wn = 0;
        for(int i = 0; i < n; ++i) {
            Pt cur = x[i], nxt = x[(i + 1) % n];
            if(cur.second <= pv.second) {
                if(nxt.second > pv.second && cross(cur,
                     nxt, pv) > 0)
                    ++wn;
            } else if(nxt.second <= pv.second && cross(
                 cur, nxt, pv) < 0) {
                --wn;
            }
        }
        return wn != 0;
    }
}
```

4.3 Closest Pair of Points

```
// Divide and Conquer untuk mencari pasangan dua titik
     2D dengan jarak Euclidean terdekat.
// O(N log N) atau O(N log^2 N).
```


New CP Leak
BINUS University

Page 10 of 23

```
#define fi first
#define se second
typedef pair<int, int> pii;

struct Point {
    int x, y, id;
};

double dist(Point p1, Point p2) {
    return hypot((double)(p1.x - p2.x), (double)(p1.y -↩
         p2.y));
}

pair<pii, double> bruteForce(Point P[], int n) {
    double mn = 1e18;
    pii ret = {-1, -1};
    for(int i = 0; i < n; ++i) {
        for(int j = i + 1; j < n; ++j) {
            double d = dist(P[i], P[j]);
            if(d < mn) {
                ret = {P[i].id, P[j].id};
                mn = d;
            }
        }
    }
    return {ret, mn};
}

pair<pii, double> getmin(pair<pii, double> a, pair<pii,↩
     double> b) {
    if(a.fi.fi == -1) return b;
    if(b.fi.fi == -1) return a;
    return (a.se < b.se) ? a : b;
}

pair<pii, double> stripClosest(vector<Point>& strip, ↩
     double d) {
    double mn = d;
    pii ret = {-1, -1};
    sort(strip.begin(), strip.end(), [](const Point& a,↩
         const Point& b) {
        return a.y < b.y;
    });
    int sz = strip.size();
    for(int i = 0; i < sz; ++i) {
        for(int j = i + 1; j < sz && (strip[j].y - ↩
             strip[i].y) < mn; ++j) {
            double cur = dist(strip[i], strip[j]);
            if(cur < mn) {
                ret = {strip[i].id, strip[j].id};
                mn = cur;
            }
        }
    }
    return {ret, mn};
}

pair<pii, double> closestUtil(Point P[], int n) {
    if(n <= 3) return bruteForce(P, n);
    int mid = n / 2;
    Point midPoint = P[mid];
    pair<pii, double> dl = closestUtil(P, mid);
    pair<pii, double> dr = closestUtil(P + mid, n - mid↩
         );
    pair<pii, double> d = getmin(dl, dr);

    vector<Point> strip;
    for(int i = 0; i < n; i++) {
        if(abs(P[i].x - midPoint.x) < d.second)
            strip.push_back(P[i]);
    }
    return getmin(d, stripClosest(strip, d.second));
}

pair<pii, double> closest(Point P[], int n) {
    sort(P, P + n, [](const Point& a, const Point& b) {
        return a.x < b.x;
    });
    return closestUtil(P, n);
}
```

4.4 Smallest Enclosing Circle

```
// Algoritma Welzl teracak untuk mencari Minimum ↩
     Enclosing Circle dari himpunan titik 2D.
// Expected O(N) dengan pengacakan titik (shuffle).
// NOTE: WAJIB shuffle titik sebelum memanggil welzl: ↩
     shuffle(p.begin(), p.end(), rng).
//       Kasus 3 titik kolinear ditangani (fallback ke ↩
     circle dari 2 titik terjauh).
struct Point {
  double x, y;
};

struct Circle {
  double x, y, r;
  Circle() : x(0), y(0), r(-1) {}
  Circle(double _x, double _y, double _r): x(_x), y(_y)↩
       , r(_r) {}
};

Circle trivial(const vector<Point>& r) {
  if(r.size() == 0)
    return Circle(0, 0, -1);
  else if(r.size() == 1)
    return Circle(r[0].x, r[0].y, 0);
  else if(r.size() == 2) {
    double cx = (r[0].x + r[1].x) / 2.0, cy = (r[0].y +↩
         r[1].y) / 2.0;
    double rad = hypot(r[0].x - r[1].x, r[0].y - r[1].y↩
         ) / 2.0;
    return Circle(cx, cy, rad);
  } else {
    double x0 = r[0].x, x1 = r[1].x, x2 = r[2].x;
    double y0 = r[0].y, y1 = r[1].y, y2 = r[2].y;
    double d = (x0 - x2) * (y1 - y2) - (x1 - x2) * (y0 ↩
         - y2);
    // Jika 3 titik kolinear (d ≈ 0), cari 2 titik ↩
         terjauh lalu buat circle dari keduanya
    if(fabs(d) < 1e-9) {
      double d01 = hypot(x0-x1, y0-y1);
      double d02 = hypot(x0-x2, y0-y2);
      double d12 = hypot(x1-x2, y1-y2);
      if(d01 >= d02 && d01 >= d12)
        return Circle((x0+x1)/2, (y0+y1)/2, d01/2);
      else if(d02 >= d12)
        return Circle((x0+x2)/2, (y0+y2)/2, d02/2);
      else
        return Circle((x1+x2)/2, (y1+y2)/2, d12/2);
    }
    double cx = (((x0 - x2) * (x0 + x2) + (y0 - y2) * (↩
         y0 + y2)) / 2 *
                (y1 - y2) - ((x1 - x2) * (x1 + x2) + (↩
                     y1 - y2) * (y1 + y2)) / 2
                * (y0 - y2)) / d;
    double cy = (((x1 - x2) * (x1 + x2) + (y1 - y2) * (↩
         y1 + y2)) / 2 *
                (x0 - x2) - ((x0 - x2) * (x0 + x2) + (↩
                     y0 - y2) * (y0 + y2)) / 2
                * (x1 - x2)) / d;
    return Circle(cx, cy, hypot(x0 - cx, y0 - cy));
  }
}

// SHUFFLE POINTS FIRST: shuffle(p.begin(), p.end(), ↩
     rng)
Circle welzl(const vector<Point>& p, int idx = 0, ↩
     vector<Point> r = {}) {
  if(idx == (int) p.size() || r.size() == 3)
    return trivial(r);
  Circle d = welzl(p, idx + 1, r);
  if(hypot(p[idx].x - d.x, p[idx].y - d.y) > d.r) {
    r.push_back(p[idx]);
    d = welzl(p, idx + 1, r);
  }
  return d;
}
```

4.5 Sutherland-Hodgman Algorithm

```
// Sutherland-Hodgman Polygon Clipping untuk memotong ↩
     poligon terhadap poligon konveks/half-plane.
// O(N * M) dengan N titik poligon dan M titik pemotong↩
     .
const double EPS = 1e-9;

struct point {
    double x, y;
    point(double _x = 0, double _y = 0) : x(_x), y(_y) ↩
         {}
};

struct vec {
    double x, y;
    vec(double _x = 0, double _y = 0) : x(_x), y(_y) {}
    vec(point a, point b) : x(b.x - a.x), y(b.y - a.y) ↩
         {}
};

double cross(vec a, vec b) { return a.x * b.y - a.y * b↩
     .x; }
bool ccw(point p, point q, point r) { return cross(vec(↩
     p, q), vec(p, r)) > EPS; }

point intersect(point s1, point e1, point s2, point e2)↩
     {
    double num1 = (s1.x * e1.y - s1.y * e1.x) * (s2.x -↩
         e2.x) - (s1.x - e1.x) * (s2.x * e2.y - s2.y * ↩
         e2.x);
    double num2 = (s1.x * e1.y - s1.y * e1.x) * (s2.y -↩
         e2.y) - (s1.y - e1.y) * (s2.x * e2.y - s2.y * ↩
         e2.x);
    double den = (s1.x - e1.x) * (s2.y - e2.y) - (s1.y ↩
         - e1.y) * (s2.x - e2.x);
    return point(num1 / den, num2 / den);
}

void clip(vector<point>& poly_points, point p1, point ↩
     p2) {
    vector<point> new_points;
    int n = poly_points.size();
    for(int i = 0; i < n; i++) {
        int k = (i + 1) % n;
        bool i_in = cross(vec(p1, p2), vec(p1, ↩
             poly_points[i])) >= -EPS;
        bool k_in = cross(vec(p1, p2), vec(p1, ↩
             poly_points[k])) >= -EPS;
        if(i_in && k_in)
            new_points.push_back(poly_points[k]);
        else if(!i_in && k_in) {
```


New CP Leak
BINUS University
Page 11 of 23

```
            new_points.push_back(intersect(p1, p2, ↩
                poly_points[i], poly_points[k]));
            new_points.push_back(poly_points[k]);
        } else if(i_in && !k_in) {
            new_points.push_back(intersect(p1, p2, ↩
                poly_points[i], poly_points[k]));
        }
    }
    poly_points = new_points;
}

double area(const vector<point>& p) {
    double res = 0.0;
    int n = p.size();
    for(int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        res += p[i].x * p[j].y - p[j].x * p[i].y;
    }
    return fabs(res) / 2.0;
}

void suthHodgClip(vector<point>& poly_points, const ↩
     vector<point>& clipper_points) {
    int n = clipper_points.size();
    for(int i = 0; i < n; i++) {
        int k = (i + 1) % n;
        clip(poly_points, clipper_points[i], ↩
             clipper_points[k]);
    }
}
```

4.6 Centroid of Polygon

$C_x = \frac{1}{6A} \sum_{i=0}^{n-1} (x_i + x_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$
$C_y = \frac{1}{6A} \sum_{i=0}^{n-1} (y_i + y_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$

4.7 Pick Theorem

A: Area of a simply closed lattice polygon
B: Number of lattice points on the edges
I: Number of points in the interior
$A = I + \frac{B}{2} - 1$

5 String

5.1 KMP

```
// KMP untuk pencocokan string pola T di dalam teks S ↩
     dan prekomputasi tabel LPS (pi).
// Prekomputasi O(|T|), Pencarian O(|S| + |T|).
vector<int> compute_lps(const string& t) {
    int n = t.size();
    vector<int> pi(n, 0);
    for(int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while(j > 0 && t[i] != t[j])
            j = pi[j - 1];
        if(t[i] == t[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_search(const string& s, const string& t↩
     ) {
    vector<int> matches;
    if(t.empty() || s.empty()) return matches;
    vector<int> pi = compute_lps(t);
    int j = 0;
    for(int i = 0; i < (int)s.size(); i++) {
        while(j > 0 && s[i] != t[j])
            j = pi[j - 1];
        if(s[i] == t[j])
            j++;
        if(j == (int)t.size()) {
            matches.push_back(i - (int)t.size() + 1);
            j = pi[j - 1];
        }
    }
    return matches;
}
```

5.2 Aho-Corasick

```
// Aho-Corasick Automaton untuk pencarian banyak pola ↩
     string sekaligus (multi-pattern matching) dan ↩
     suffix failure links.
// Konstruksi O(total panjang pola * alfabet), ↩
     Pencarian Teks O(|T| + matches).
const int K = 26;
struct Vertex {
  int next[K];
  bool leaf = 0;
  int p = -1, ans = 0;
  char pch;
  int link = -1, mlink = -1;
  int go[K];
  Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {
    fill(begin(next), end(next), -1);
    fill(begin(go), end(go), -1);
  }
};

vector<Vertex> t;

void init_aho_corasick() {
    t.clear();
    t.emplace_back();
}

int add_string(string const& s) {
  int v = 0;
  for (char ch : s) {
    int c = ch - 'a';
    if (t[v].next[c] == -1) {
      t[v].next[c] = t.size();
      t.emplace_back(v, ch);
    }
    v = t[v].next[c];
  }
  t[v].leaf = 1;
  return v;
}

int go(int v, char ch);

int get_link(int v) {
  if (t[v].link == -1) {
    if(v == 0 || t[v].p == 0)
      t[v].link = 0;
    else
      t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}

int get_mlink(int v) {
  if (t[v].mlink == -1) {
    if(v == 0 || t[v].p == 0)
      t[v].mlink = 0;
    else {
      int l = get_link(v);
      if(l == 0) t[v].mlink = 0;
      else if(t[l].leaf) t[v].mlink = l;
      else t[v].mlink = get_mlink(l);
    }
  }
  return t[v].mlink;
}

int go(int v, char ch) {
  int c = ch - 'a';
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1)
      t[v].go[c] = t[v].next[c];
    else
      t[v].go[c] = (v == 0 ? 0 : go(get_link(v), ch));
  }
  return t[v].go[c];
}
```

5.3 Manacher's Algorithm

```
// Algoritma Manacher untuk mencari seluruh substring ↩
     palindrom (panjang ganjil & genap) dan mengecek ↩
     isPalindrome dalam O(1).
// Prekomputasi O(N), Query isPalindrome O(1).
#define rep(i, a, b) for(int i = (a); i <= (b); ++i)

const int MAX = 1000005;
int lps[MAX * 2];

inline void manacher(const string &str){
  rep(i, 0, 2 * (int)str.size()) lps[i] = 0;
  lps[0] = 0;
  lps[1] = 1;
  int l, r, j, k;
  rep(i, 2, 2 * (int)str.size()){
    l = (i>>1) - (lps[i]>>1);
    r = ((i-1)>>1) + (lps[i]>>1);
    while(1){
      if (l == 0 || r+1 == (int)str.size()) break;
      if (str[l-1] != str[r+1]) break;
      l--, r++;
    }
    lps[i] = r-l+1;
    if (lps[i] > 2){
      j = i-1, k = i+1;
      while (lps[j] - j < lps[i] - i)
        lps[k++] = lps[j--];
      lps[k] = lps[i] - (i-j);
      i = k-1;
    }
  }
}

bool isPalindrome(int l, int r){
  return lps[r + l + 1] >= r - l + 1;
}
```

5.4 Z Algorithm

```
// Z-Algorithm menghitung array Z di mana Z[i] adalah ↩
     panjang prefix terpanjang dari S yang cocok dengan ↩
     S[i...|S|-1].
```


New CP Leak
BINUS University

Page 12 of 23

```
// O(N) linier.
typedef vector<int> vi;

vi Z(const string &S){
  vi z(S.size());
  int l = -1, r = -1;
  for (int i = 1; i < (int)S.size(); i++){
    z[i] = i >= r ? 0 : min(r-i, z[i-l]);
    while(i + z[i] < (int)S.size() && S[i+z[i]] == S[z[↩
        i]])
      z[i]++;
    if (i + z[i] > r) l = i, r = i+z[i];
  }
  return z;
}
```

5.5 Hashing

```
// Double Polynomial Rolling Hash untuk menghitung dan ↩
    membandingkan hash substring dalam O(1).
// Prekomputasi O(N), Query getHash O(1).
// NOTE: Base (key) bersifat static, di-generate sekali↩
    saja. Semua instance hashing berbagi
//       base yang sama sehingga hash bisa dibandingkan↩
    antar string berbeda.
//       Gunakan double hash (2 moduli) untuk ↩
    mengurangi collision probability.
typedef long long ll;

mt19937 rng_hash(chrono::steady_clock::now().↩
    time_since_epoch().count());

class hashing{
public:
  int n;
  string s;
  static constexpr int mod[2] = {1000000007, ↩
      1000000009};
  static vector<ll> key[2];
  static bool key_initialized;
  vector<pair<ll,ll>> hsh;

  static void init_keys(int sz) {
    if(key_initialized && (int)key[0].size() >= sz + 1)↩
        return;
    for(int i = 0; i < 2; i++){
      key[i].resize(sz + 1);
      if(!key_initialized) {
        key[i][0] = 1;
        do {
          key[i][1] = 257 + (rng_hash() % (mod[i] - ↩
              300));
        } while(key[i][1] <= 256);
      }
      int start = key_initialized ? (int)key[i].size() ↩
          : 2;
      key[i].resize(sz + 1);
      for (int j = start; j <= sz; j++){
        key[i][j] = 1LL * key[i][j-1] * key[i][1] % mod↩
            [i];
      }
    }
    key_initialized = true;
  }

  hashing(string _s) : s(_s){
    n = (int) s.size();
    if(n == 0) return;
    init_keys(n);
    hsh.resize(n);
    hsh[0] = {s[0], s[0]};
    for (int i = 1; i < n; i++){
      hsh[i].first = (1LL * hsh[i-1].first * key[0][1] ↩
          % mod[0] + s[i]) % mod[0];
      hsh[i].second= (1LL * hsh[i-1].second * key[1][1]↩
          % mod[1] + s[i]) % mod[1];
    }
  }

  pair<ll,ll> getHash(int l, int r){
    if(l > r || n == 0) return {0, 0};
    pair<ll,ll> R = hsh[r];
    if (l == 0) return R;
    pair<ll,ll> L = hsh[l-1];
    R.first = (1LL * R.first + mod[0] - 1LL * L.first *↩
        key[0][r-l+1] % mod[0]) % mod[0];
    R.second = (1LL * R.second + mod[1] - 1LL * L.↩
        second * key[1][r-l+1] % mod[1]) % mod[1];
    return R;
  }
};
vector<ll> hashing::key[2];
bool hashing::key_initialized = false;
```

6 Math

6.1 Extended Euclidean GCD

```
// Extended Euclidean Algorithm untuk menghitung gcd(a,↩
    b) serta koefisien Bezout x, y (ax + by = gcd(a, b↩
    )).
// O(log(min(a, b))).
typedef long long ll;

tuple<ll, ll, ll> gcd(ll a, ll b) {
  if(b == 0) return {a, 1, 0};
  auto [d, x1, y1] = gcd(b, a % b);
  return {d, y1, x1 - y1 * (a / b)};
}
```

6.2 Generalized CRT

```
// Generalized CRT untuk menyelesaikan sistem ↩
    kongruensi linear dengan modulo yang tidak harus ↩
    prima relatif.
// O(log(min(n, m))) per pasang persamaan.
// NOTE: LCM dihitung sebagai (m/gcd)*n untuk ↩
    menghindari overflow.
//       Untuk banyak persamaan, panggil CRT secara ↩
    berulang secara pair-wise.
template<typename T>
T extended_euclid(T a, T b, T& x, T& y) {
  if(b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  T xx, yy, gcd;
  gcd = extended_euclid(b, a % b, xx, yy);
  x = yy;
  y = xx - (yy * (a / b));
  return gcd;
}

template<typename T>
T MOD(T a, T b) {
  return (a % b + b) % b;
}

template<typename T>
pair<T, T> CRT(T a, T n, T b, T m) {
  T _n, _m;
  T gcd = extended_euclid(n, m, _n, _m);
  if(n == m) {
    if(a == b)
      return pair<T, T>(a, n);
    else
      return pair<T, T>(-1, -1);
  } else if(abs(a - b) % gcd != 0)
    return pair<T, T>(-1, -1);
  else {
    T lcm = (m / gcd) * n;
    T x = MOD(a + MOD(n * MOD(_n * ((b - a) / gcd), m /↩
        gcd), lcm), lcm);
    return pair<T, T>(x, lcm);
  }
}
```

6.3 Generalized Lucas Theorem

```
// Generalized Lucas Theorem untuk menghitung kombinasi↩
    C(n, r) modulo prime power (p^k).
// O(p^k + log_p n).
typedef long long ll;

ll fast(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while(b > 0) {
        if(b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}

ll totient(ll n) {
    ll result = n;
    for(ll p = 2; p * p <= n; ++p) {
        if(n % p == 0) {
            while(n % p == 0) n /= p;
            result -= result / p;
        }
    }
    if(n > 1) result -= result / n;
    return result;
}

ll E(ll n, ll p) {
    ll tot = 0;
    while(n != 0) {
        tot += n / p;
        n /= p;
    }
    return tot;
}

ll F(ll n, ll p, ll pk, const vector<ll>& fctp) {
    ll ans = 1;
    while(n > 0) {
        ans = (__int128)ans * fast(fctp[pk], n / pk, pk↩
            ) % pk * fctp[n % pk] % pk;
        n /= p;
    }
    return ans;
}

ll prime_power_lucas(ll n, ll r, ll p, ll pk) {
```


New CP Leak
BINUS University
Page 13 of 23

```
    if(r < 0 || r > n) return 0;
    vector<ll> fctp(pk + 1, 1);
    for(ll i = 1; i <= pk; ++i) {
        fctp[i] = fctp[i - 1];
        if(i % p != 0) fctp[i] = (fctp[i] * (i % pk)) %↩
             pk;
    }

    ll power = E(n, p) - E(n - r, p) - E(r, p);
    if(power >= 60 || fast(p, power, pk) == 0) return ↩
         0;

    ll top = fast(p, power, pk) * F(n, p, pk, fctp) % ↩
         pk;
    ll bot = F(r, p, pk, fctp) * F(n - r, p, pk, fctp) ↩
         % pk;
    ll bot_inv = fast(bot, totient(pk) - 1, pk);
    return (top * bot_inv) % pk;
}
```

6.4 Linear Diophantine

```
// Menyelesaikan persamaan Diophantine linear ax + by =↩
     c dan mencari solusi integer yang meminimalkan |x|↩
     + |y|.
// O(log(min(a, b))).
// NOTE: Menangani edge case b=0 (atau a=0) secara ↩
    terpisah.
//       Jika c % gcd(a,b) != 0, tidak ada solusi (↩
    return -1).
typedef long long ll;

ll extgcd(ll a, ll b, ll &x, ll &y) {
    if(b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0↩
    , ll &g) {
    g = extgcd(abs(a), abs(b), x0, y0);
    if(c % g != 0) return false;
    x0 *= c / g;
    y0 *= c / g;
    if(a < 0) x0 = -x0;
    if(b < 0) y0 = -y0;
    return true;
}

ll min_abs_sum(ll a, ll b, ll c) {
    ll x0, y0, g;
    if(!find_any_solution(a, b, c, x0, y0, g)) return ↩
         -1;
    // Edge case: jika b == 0, solusi hanya x0 (tidak ↩
         bisa geser)
    if(b == 0) return abs(x0) + abs(y0);
    // Edge case: jika a == 0, solusi hanya y0 (tidak ↩
         bisa geser)
    if(a == 0) return abs(x0) + abs(y0);
    ll step_x = b / g, step_y = a / g;
    ll k = -x0 / step_x;
    ll ans = LLONG_MAX;
    for(ll i = k - 2; i <= k + 2; ++i) {
        ll cur_x = x0 + i * step_x;
        ll cur_y = y0 - i * step_y;
        ans = min(ans, abs(cur_x) + abs(cur_y));
    }
    return ans;
}
```

6.5 Modular Linear Equation

```
// Mencari semua solusi kongruensi linear ax = b (mod n↩
    ).
// O(log n + gcd(a, n)).
typedef vector<int> vi;

int ext_gcd(int a, int b, int &x, int &y) {
    if(b == 0) { x = 1; y = 0; return a; }
    int x1, y1;
    int g = ext_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

inline int mod_val(long long a, int m) {
    return (a % m + m) % m;
}

vi modular_linear_equation_solver(int a, int b, int n) ↩
    {
  int x, y;
  vi ret;
  int g = ext_gcd(a, n, x, y);
  if(b % g == 0) {
    int x0 = mod_val(1LL * x * (b / g), n);
    for(int i = 0; i < g; i++)
      ret.push_back(mod_val(x0 + 1LL * i * (n / g), n))↩
          ;
  }
  return ret;
}
```

6.6 Miller-Rabin and Pollard's Rho

```
// Uji keprimaan Miller-Rabin deterministik (hingga ↩
    2^64) dan faktorisasi prima Pollard's Rho.
// Miller-Rabin O(k log N) (k=12), Pollard's Rho O(N↩
    ^(1/4) log N).
typedef long long ll;
typedef unsigned __int128 u128;

ll fast(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while(b > 0) {
        if(b & 1) res = (u128)res * a % mod;
        a = (u128)a * a % mod;
        b >>= 1;
    }
    return res;
}

bool miller_rabin(ll n){
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;
    ll d = n - 1;
    ll s = 0;
    while (d % 2 == 0){
        s++;
        d /= 2;
    }
    vector<ll> base = {2, 3, 5, 7, 11, 13, 17, 19, 23, ↩
         29, 31, 37};
    for (auto a : base){
        if (n <= a) break;
        ll x = fast(a, d, n);
        if (x == 1 || x == n-1) continue;
        bool comp = true;
        for (ll r = 1; r < s; r++){
            x = ((u128)x * x) % n;
            if (x == n-1){
                comp = false;
                break;
            }
        }
        if (comp) return false;
    }
    return true;
}

ll pollard_rho(ll n){
    if (n % 2 == 0) return 2;
    if (n == 1) return 1;
    ll turtle = 2, hare = 2, c = 1, d = 1;
    while (d == 1){
        turtle = (((u128)turtle * turtle) + c) % n;
        hare = (((u128)hare * hare) + c) % n;
        hare = (((u128)hare * hare) + c) % n;
        d = std::gcd(n, llabs(turtle - hare));
        if (d == n){
            c++;
            turtle = 2;
            hare = 2;
            d = 1;
        }
    }
    return d;
}

void factor(ll n, vector<ll> &ans){
    if (n > 1){
        if (miller_rabin(n)){
            ans.push_back(n);
            return;
        }
        ll x = pollard_rho(n);
        factor(x, ans);
        factor(n/x, ans);
    }
}
```

6.7 Berlekamp-Massey

```
// Algoritma Berlekamp-Massey untuk mencari relasi ↩
    rekurensi linear terpendek dan suku ke-K barisan ↩
    via eksponensiasi polinomial.
// Berlekamp-Massey O(N^2), Perhitungan suku ke-K O(M^2↩
    log K) dengan M panjang rekurensi.
#define pb push_back
typedef long long ll;
#define SZ 233333
const int MOD = 1e9 + 7;

ll qp(ll a, ll b) {
  ll x = 1;
  a %= MOD;
  while(b) {
    if(b & 1)
      x = x * a % MOD;
    a = a * a % MOD;
```


New CP Leak
BINUS University
Page 14 of 23

```
    b >>= 1;
  }
  return x;
}

namespace linear_seq {
  vector<int> BM(vector<int> x) {
    vector<int> ls, cur;
    int lf = -1, ld = -1;
    for(int i = 0; i < int(x.size()); ++i) {
      ll t = 0;
      for(int j = 0; j < int(cur.size()); ++j)
        t = (t + x[i - j - 1] * (ll)cur[j]) % MOD;
      if((t - x[i]) % MOD == 0) continue;
      if(!cur.size()) {
        cur.resize(i + 1);
        lf = i;
        ld = (t - x[i]) % MOD;
        continue;
      }
      ll k = -(x[i] - t) * qp(ld, MOD - 2) % MOD;
      vector<int> c(i - lf - 1);
      c.pb(k);
      for(int j = 0; j < int(ls.size()); ++j)
        c.pb(-ls[j]*k % MOD);
      if(c.size() < cur.size())
        c.resize(cur.size());
      for(int j = 0; j < int(cur.size()); ++j)
        c[j] = (c[j] + cur[j]) % MOD;
      if(i - lf + (int)ls.size() >= (int)cur.size())
        ls = cur, lf = i, ld = (t - x[i]) % MOD;
      cur = c;
    }
    for(int i = 0; i < int(cur.size()); ++i)
      cur[i] = (cur[i] % MOD + MOD) % MOD;
    return cur;
  }

  int m;
  ll a[SZ], h[SZ], t_[SZ], s[SZ], t[SZ];

  void mull(ll* p, ll* q) {
    for(int i = 0; i < m + m; ++i)
      t_[i] = 0;
    for(int i = 0; i < m; ++i) if(p[i])
        for(int j = 0; j < m; ++j)
          t_[i + j] = (t_[i + j] + p[i] * q[j]) % MOD;
    for(int i = m + m - 1; i >= m; --i) if(t_[i])
        for(int j = m - 1; ~j; --j)
          t_[i - j - 1] = (t_[i - j - 1] + t_[i] * h[j↩
              ]) % MOD;
    for(int i = 0; i < m; ++i)
      p[i] = t_[i];
  }

  ll calc(ll K) {
    for(int i = m; ~i; --i)
      s[i] = t[i] = 0;
    s[0] = 1;
    if(m != 1)
      t[1] = 1;
    else
      t[0] = h[0];
    while(K) {
      if(K & 1)
        mull(s, t);
      mull(t, t);
      K >>= 1;
    }
    ll su = 0;
    for(int i = 0; i < m; ++i)
      su = (su + s[i] * a[i]) % MOD;
    return (su % MOD + MOD) % MOD;
  }

  int work(vector<int> x, ll n) {
    if(n < int(x.size()))
      return x[n];
    vector<int> v = BM(x);
    m = v.size();
    if(!m)
      return 0;
    for(int i = 0; i < m; ++i)
      h[i] = v[i], a[i] = x[i];
    return calc(n);
  }
}
using linear_seq::work;
```

6.8 Fast Fourier Transform

```
// FFT bilangan kompleks untuk perkalian polinomial ↩
    berpresisi tinggi.
// O(N log N) dengan N ukuran derajat polinomial hasil ↩
    pembulatan ke pangkat 2.
using ll = long long;
using ld = double;
using cd = complex<ld>;
const ld PI = acos(-(ld)1);

void fft(vector<cd> &a, ll sign = 1) {
    ll n = a.size();

    for (ll i = 1, j = 0; i < n; i++) {
        ll bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
            swap(a[i], a[j]);
        }
    }

    for (ll len = 2; len <= n; len <<= 1) {
        ld theta = sign * 2 * PI / len;
        cd wlen(cos(theta), sin(theta));
        for (ll i = 0; i < n; i += len) {
            cd w(1);
            for (ll j = 0; j < len / 2; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }

    if (sign == -1) {
        for (cd &x : a) {
            x /= n;
        }
    }
}

vector<ll> multiply (vector<ll> &a, vector<ll> &b){
    vector<cd> fa(a.begin(), a.end());
    vector<cd> fb(b.begin(), b.end());
    ll n = 1;
    while (n < (a.size() + b.size())){
        n <<= 1;
    }
    fa.resize(n);
    fb.resize(n);
    fft(fa);
    fft(fb);

    for (ll i = 0; i < n; i++){
        fa[i] *= fb[i];
    }

    fft(fa, -1);

    vector<ll> res(n);
    for (ll i = 0; i < n; i++){
        res[i] = round(fa[i].real());
    }
    return res;
}
```

6.9 Number Theoretic Transform

```
// NTT untuk perkalian polinomial eksak modulo prima ↩
    64-bit yang mendukung NTT.
// O(N log N).
namespace FFT {
  const int LN = 23;
  const int N = 1 << LN;
  typedef long long LL;
  const LL MOD = 9223372036737335297;
  const LL PRIMITIVE_ROOT = 3;

  LL root[N];

  inline LL power(LL x, LL y) {
    LL ret = 1;
    for(; y; y >>= 1) {
      if(y & 1)
        ret = (__int128) ret * x % MOD;
      x = (__int128) x * x % MOD;
    }
    return ret;
  }

  inline void init_fft() {
    const LL UNITY = power(PRIMITIVE_ROOT, (MOD - 1) >>↩
        LN);
    root[0] = 1;
    for(int i = 1; i < N; i++)
      root[i] = (__int128) UNITY * root[i - 1] % MOD;
  }

  inline void fft(int n, vector<LL>& a, bool invert) {
    for(int i = 1, j = 0; i < n; ++i) {
      int bit = n >> 1;
      for(; j >= bit; bit >>= 1)
        j -= bit;
      j += bit;
      if(i < j)
        swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <<= 1) {
      LL wlen = (invert ? root[N - N / len] : root[N / ↩
          len]);
      for(int i = 0; i < n; i += len) {
        LL w = 1;
        for(int j = 0; j < (len >> 1); j++) {
          LL u = a[i + j];
          LL v = (__int128) a[i + j + len / 2] * w % ↩
              MOD;
```


New CP Leak
BINUS University
Page 15 of 23

```
        a[i + j] = ((__int128) u + v) % MOD;
        a[i + j + len / 2] = ((__int128) u - v + MOD)%MOD;
        w = (__int128) w * wlen % MOD;
      }
    }
  }
  if(invert) {
    LL inv = power(n, MOD - 2);
    for(int i = 0; i < n; i++)
      a[i] = (__int128) a[i] * inv % MOD;
  }
}

inline vector<LL> multiply(vector<LL> a, vector<LL> b) {
  vector<LL> c;
  if(a.empty() || b.empty()) return c;
  int len = 1;
  while(len < (int)(a.size() + b.size() - 1)) len <<= 1;
  a.resize(len, 0);
  b.resize(len, 0);
  fft(len, a, false);
  fft(len, b, false);
  c.resize(len);
  for(int i = 0; i < len; ++i)
    c[i] = (__int128) a[i] * b[i] % MOD;
  fft(len, c, true);
  return c;
}
}
```

6.10 Gauss-Jordan

```
// Eliminasi Gauss-Jordan dengan full pivoting untuk menyelesaikan sistem persamaan linear AX = B, inversi matriks, dan determinan.
// Kompleksitas Waktu: O(N^3).
const double EPS = 1e-10;
typedef vector<int> VI;
typedef double T;
typedef vector<T> VT;
typedef vector<VT> VVT;

T GaussJordan(VVT& a, VVT& b) {
  const int n = a.size();
  const int m = b[0].size();
  VI irow(n), icol(n), ipiv(n);
  T det = 1;
  for(int i = 0; i < n; i++) {
    int pj = -1, pk = -1;
    for(int j = 0; j < n; j++) if(!ipiv[j])
      for(int k = 0; k < n; k++) if(!ipiv[k])
        if(pj == -1 || fabs(a[j][k]) > fabs(a[pj][pk])) {
          pj = j;
          pk = k;
        }
    if(fabs(a[pj][pk]) < EPS) {
      return 0;
    }
    ipiv[pk]++;
    swap(a[pj], a[pk]);
    swap(b[pj], b[pk]);
    if(pj != pk)
      det *= -1;
    irow[i] = pj;
    icol[i] = pk;
    T c = 1.0 / a[pk][pk];
    det *= a[pk][pk];
    a[pk][pk] = 1.0;
    for(int p = 0; p < n; p++)
      a[pk][p] *= c;
    for(int p = 0; p < m; p++)
      b[pk][p] *= c;
    for(int p = 0; p < n; p++) if(p != pk) {
        c = a[p][pk];
        a[p][pk] = 0;
        for(int q = 0; q < n; q++)
          a[p][q] -= a[pk][q] * c;
        for(int q = 0; q < m; q++)
          b[p][q] -= b[pk][q] * c;
      }
  }
  for(int p = n - 1; p >= 0; p--) if(irow[p] != icol[p]) {
      for(int k = 0; k < n; k++)
        swap(a[k][irow[p]], a[k][icol[p]]);
    }
  return det;
}
```

6.11 Josephus Problem

```
// Masalah eliminasi Josephus (0-indexed) untuk mencari posisi orang terakhir yang bertahan.
// josephus_fast O(k log n), josephus_linear O(n).
typedef long long ll;

ll josephus_fast(ll n, ll k) {
    if(n == 1) return 0;
    if(k == 1) return n - 1;
    if(k > n) return (josephus_fast(n - 1, k) + k) % n;
    ll cnt = n / k;
    ll res = josephus_fast(n - cnt, k);
    res -= n % k;
    if(res < 0) res += n;
    else res += res / (k - 1);
    return res;
}

int josephus_linear(int n, int k) {
    int res = 0;
    for(int i = 1; i <= n; ++i)
        res = (res + k) % i;
    return res;
}
```

6.12 Derangement

```
typedef long long ll;
const int N = 100005;
const ll MOD = 1e9 + 7;
ll der[N];

void compute_derangements(int n) {
    der[0] = 1;
    der[1] = 0;
    for(int i = 2; i <= n; ++i)
        der[i] = (ll)(i - 1) % MOD * ((der[i - 1] + der[i - 2]) % MOD) % MOD;
}
```

6.13 Bernoulli Number

$$\sum_{k=1}^{n} k^m = \frac{1}{m+1}\sum_{i=0}^{m}\binom{m+1}{i}B_i^+ n^{m+1-i}$$

$$\sum_{k=1}^{n} k^m = m!\sum_{i=0}^{m}\frac{B_i^+ n^{m+1-i}}{i!(m+1-i)!}$$

$$B_n^+ = 1 - \sum_{i=0}^{n-1}\binom{n}{i}\frac{B_i^+}{n-i+1}, B_0^+ = 1$$

6.14 Forbenius Number

Teorema Koin Frobenius untuk dua bilangan bulat relatif prima X dan Y.
Nilai terbesar tak dapat dibentuk: $XY - X - Y$.
Jumlah nilai yang tidak dapat dibentuk: $(X-1)(Y-1)/2$.

6.15 Stars and Bars with Upper Bound

$P = (1 - X^{r_1+1})\dots(1 - X^{r_n+1}) = \sum_i c_i X^{e_i}$
$Ans = \sum_i c_i \binom{N-e_i+n-1}{n-1}$

6.16 Arithmetic Sequences

$$U_n = a + (n-1)a_1 + \frac{(n-1)(n-2)}{1\times 2}a_2 + \dots + \frac{(n-1)(n-2)(n-3)\dots}{1\times 2\times 3\times\dots}a_r$$

$$S_n = n \times a + \frac{n(n-1)}{1\times 2}a_1 + \frac{n(n-1)(n-2)}{1\times 2\times 3}a_2 + \dots + \frac{n(n-1)(n-2)(n-3)\dots}{1\times 2\times 3\dots}a_r$$

6.17 Mobius Function

For any positive integer n,

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ 0 & \text{if } a^2 \mid n \text{ for some } a > 1, \\ (-1)^k & \text{if } n \text{ is the product of } k \text{ distinct primes.} \end{cases} \square$$

```
const int N = 1e6;
int mobius[N + 1];
vector<int> primes;
bool is_prime[N + 1];

// Linear Sieve O(N)
void precompute_mobius() {
  fill(is_prime, is_prime + N + 1, true);
  is_prime[0] = is_prime[1] = false;
  mobius[1] = 1;

  for (int i = 2; i <= N; i++) {
    if (is_prime[i]) {
      primes.push_back(i);
      mobius[i] = -1;
    }
    for (int p : primes) {
      if (i * p > N) break;
      is_prime[i * p] = false;
      if (i % p == 0) {
        mobius[i * p] = 0;
        break;
      } else {
        mobius[i * p] = -mobius[i];
```


New CP Leak
BINUS University
Page 16 of 23

```
        }
      }
    }
  }
```

6.18 Floor Sum

```
// Algoritma/Fungsi: Menghitung jumlahan floor((a*i + b) / m) untuk i = 0...n-1 dalam O(log(min(m, a))).
// Kompleksitas Waktu: O(log(min(m, a))).
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll sum_of_floor(ll n, ll m, ll a, ll b){
  ll ans = 0;
  if (a >= m){
    ans += (n % 2 == 0 ? (n / 2) * (n - 1) : n * ((n - 1) / 2)) * (a / m);
    a %= m;
  }
  if(b >= m){
    ans += n * (b / m);
    b %= m;
  }
  ll y_max = (a * n + b) / m, x_max = (y_max * m - b);
  if (y_max == 0) return ans;
  ans += (n - (x_max + a - 1) / a) * y_max;
  ans += sum_of_floor(y_max, a, m, (a - x_max % a) % a);
  return ans;
}
```

6.19 Combin Identity

1. $\sum_{0 \le k \le n} \binom{n-k}{k} = Fib_{n+1}$
2. $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
3. $k\binom{n}{k} = n\binom{n-1}{k-1}$
4. $\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$
5. $\sum_{i=0}^{n} \binom{n}{i} = 2^n$
6. $\sum_{i \ge 0} \binom{n}{2i} = 2^{n-1}$
7. $\sum_{i \ge 0} \binom{n}{2i+1} = 2^{n-1}$
8. $\sum_{i=0}^{k} (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
9. $\sum_{i=0}^{k} \binom{n+i}{i} = \sum_{i=0}^{k} \binom{n+i}{n} = \binom{n+k+1}{k}$
10. $\sum_{i=1}^{n} i\binom{n}{i} = n2^{n-1}$
11. $\sum_{i=1}^{n} i^2\binom{n}{i} = (n+n^2)2^{n-2}$
12. Vandermonde's Identify: $\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
13. Hockey-Stick Identify:
    $n, r \in N, n > r, \sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$
14. $\sum_{i=0}^{k} \binom{k}{i}^2 = \binom{2k}{k}$
15. $\sum_{k=0}^{n} \binom{n}{k}\binom{n}{n-k} = \binom{2n}{n}$
16. $\sum_{k=q}^{n} \binom{n}{k}\binom{k}{q} = 2^{n-q}\binom{n}{q}$
17. $\sum_{i=0}^{n} k^i\binom{n}{i} = (k+1)^n$
18. $\sum_{i=0}^{n} \binom{2n}{i} = 2^{2n-1} + \frac{1}{2}\binom{2n}{n}$
19. $\sum_{i=1}^{n} \binom{n}{i}\binom{n-1}{i-1} = \binom{2n-1}{n-1}$
20. $\sum_{i=0}^{n} \binom{2n}{i}^2 = \frac{1}{2}\left(\binom{4n}{2n} + \binom{2n}{n}^2\right)$
21. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is
$$\frac{n!}{(k!)^{\frac{n}{k}}(\frac{n}{k})!} = \prod_{n \ge k}^{n-=k} \binom{n-1}{k-1}$$
22. $\sum_{i=1,3,5,\ldots}^{i \le n} \binom{n}{i} a^{n-i}b^i = \frac{1}{2}((a+b)^n - (a-b)^n)$
23. $\sum_{i=0}^{n} \frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$

6.20 Fibonacci Identity

1. $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$
2. $F_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n$
3. $\sum_{i=1}^{n} F_i = F_{n+2} - 1$
4. $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$
5. $\sum_{i=1}^{n} F_{2i} = F_{2n+1} - 1$
6. $\sum_{i=1}^{n} F_i^2 = F_n F_{n+1}$
7. $F_m F_{n+1} - F_{m-1}F_n = (-1)^n F_{m-n}$
8. $F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n(F_{n+1} + F_{n-1})$
9. $F_m F_n + F_{m-1}F_{n-1} = F_{m+n-1}$
10. $F_m F_{n+1} + F_{m-1}F_n = F_{m+n}$
11. A number is Fibonacci if and only if one or both of $(5 \cdot n^2 + 4)$ or $(5 \cdot n^2 - 4)$ is a perfect square.
12. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k
13. $gcd(F_m, F_n) = F_{gcd(m,n)}$
14. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n,
    $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}), gcd(F_{n+1}, F_{n+2}) = 1$
15. The nth Pisano period, written as $\pi(n)$, is the period with which the sequence of Fibonacci numbers taken modulo n repeats.
    $\pi(mn) = \text{lcm}(\pi(m), \pi(n))$, with m and n coprime
    If $n = 2^k$, then $\pi(n) = 3 \cdot 2^{k-1} = \frac{3n}{2}$
    If $n = 5^k$, then $\pi(n) = 4 \cdot 5^k = 4n$

6.21 Other Identity

1. Euclid's formula is a fundamental formula for generating Pythagorean
   triples given an arbitrary pair of integers m and n with $m > n > 0$.
   The formula states that the integers
   $$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$$
   form a Pythagorean triple. The triple generated by Euclid's formula is primitive
   if and only if m and n are coprime and not both odd. When both m and n are odd,
   then a, b, and c will be even, and the triple will not be primitive;
   however, dividing a, b, and c by 2 will yield a primitive triple when m and n
   are coprime and both odd.
2. The following will generate all Pythagorean triples uniquely:
   $$a = k \cdot (m^2 - n^2), b = k \cdot (2mn), c = k \cdot (m^2 + n^2)$$
   where m, n, and k are positive integers with $m > n$, and with m and n coprime
   and not both odd.
3. $a + b = a \oplus b + 2(a\&b)$.
4. $a + b = a|b + a\&b$
5. $2max(a, b) = (a + b) + |a - b|$
6. $2min(a, b) = (a + b) - |a - b|$
7. $a \oplus b = a|b - a\&b$
8. k_{th} bit (0-indexed) is set in x iff $x \bmod 2^{k+1} \ge 2^k$.
9. k_{th} bit (0-indexed) is set in x iff $(x \bmod 2^{k+1}) - (x \bmod 2^k) = 2^k$.
10. $n \bmod 2^i = n\&(2^i - 1)$
11. $1 \oplus 2 \oplus 3 \oplus \cdots \oplus (4k-1) = 0$ for any $k \ge 0$

New CP Leak
BINUS University
Page 17 of 23

6.22 Lagrange Interpolation

Bentuk Umum (Arbitrary Points):

$$P(x) = \sum_{i=0}^{k} y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Polinomial Basis Lagrange:

$$L_i(x) = \prod_{\substack{0 \leq j \leq k \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

Titik Berjarak Seragam $x_i = i$:

$$P(x) = \sum_{i=0}^{k} y_i \cdot \frac{\prod_{j=0}^{k}(x-j)}{(x-i)} \cdot \frac{(-1)^{k-i}}{i!\,(k-i)!}$$

Prefix-Suffix Decomposition:

$$\text{pref}_i = \prod_{j=0}^{i}(x-j), \quad \text{suff}_i = \prod_{j=i}^{k}(x-j)$$

$$P(x) = \sum_{i=0}^{k} y_i \cdot \frac{\text{pref}_{i-1} \cdot \text{suff}_{i+1}}{i!\,(k-i)!} \cdot (-1)^{k-i}$$

6.23 Discrete Root and Logarithm

```cpp
using ll = long long;

// Eksponensiasi Modular: (base^exp) % mod
ll power(ll base, ll exp, ll mod) {
    ll res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod;
        exp >>= 1;
    }
    return res;
}

// Extended GCD: a*x + b*y = gcd(a, b)
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Invers Modular: a^(-1) mod m
ll mod_inv(ll a, ll m) {
    ll x, y;
    ll g = extgcd(a, m, x, y);
    if (g != 1) return -1;
    return (x % m + m) % m;
}

// ↩
    ==========

// KASUS 2: Discrete Logarithm (a^x = b mod m)
// Mendukung m sembarang (Extended BSGS)
// Mengembalikan nilai x non-negatif terkecil, atau -1 ↩
    jika tidak ada solusi.
// Kompleksitas: O(sqrt(m))
// ↩
    ==========

ll solve_discrete_log(ll a, ll b, ll m) {
    a %= m; b %= m;
    if (m == 1) return 0;
    if (b == 1) return 0;

    ll add = 0, k = 1;
    ll g = std::gcd(a, m);

    while (g > 1) {
        if (b == k) return add;
        if (b % g != 0) return -1;

        b /= g;
        m /= g;
        k = (__int128)k * (a / g) % m;
        add++;
        g = std::gcd(a, m);
    }

    // Standard BSGS pada sub-masalah: (a^x) * k = b (↩
        mod m) dengan gcd(a, m) = 1
    ll n = sqrt(m) + 1;
    unordered_map<ll, ll> vals;

    // Baby-step: vals[k * a^q mod m] = q
    ll cur = k;
    for (ll q = 0; q <= n; ++q) {
        vals[cur] = q;
        cur = (__int128)cur * a % m;
    }

    // Giant-step: periksa (a^n)^p mod m
    ll an = power(a, n, m);
    cur = 1;
    for (ll p = 1; p <= n; ++p) {
        cur = (__int128)cur * an % m;
        // Ingin mencari cur * a^(-q) = b/k mod m <=> ↩
            cur = (k * a^q) mod m
        // Kita bandingkan jika cur cocok dengan tabel ↩
            baby-step yang dinormalisasi dengan b
        // Lebih sederhana: cari cur * inv(b) di tabel ↩
            baby-step
    }

    // Pendekatan alternatif tanpa invers per giant ↩
        step:
    // a^(p*n - q) * k = b <=> (a^n)^p * k = b * a^q
    vals.clear();
    ll cur_b = b;
    for (ll q = 0; q <= n; ++q) {
        vals[cur_b] = q;
        cur_b = (__int128)cur_b * a % m;
    }

    cur = k;
    for (ll p = 1; p <= n + 1; ++p) {
        cur = (__int128)cur * an % m;
        if (vals.count(cur)) {
            ll ans = p * n - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}

// ↩
    ==========

// KASUS 1: Discrete Root (x^a = b mod p) dengan p ↩
    bilangan prima
// Mengembalikan semua solusi x mod p dalam vector ↩
    terurut
// Kompleksitas: O(sqrt(p) log p)
// ↩
    ==========

// Mencari generator / Primitive Root modulo p prima
ll find_primitive_root(ll p) {
    if (p == 2) return 1;
    vector<ll> fact;
    ll phi = p - 1, temp = phi;

    for (ll d = 2; d * d <= temp; ++d) {
        if (temp % d == 0) {
            fact.push_back(d);
            while (temp % d == 0) temp /= d;
        }
    }
    if (temp > 1) fact.push_back(temp);

    for (ll g = 2; g < p; ++g) {
        bool ok = true;
        for (ll factor : fact) {
            if (power(g, phi / factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
    return -1;
}

vector<ll> solve_discrete_root(ll a, ll b, ll p) {
    b %= p;
    if (b == 0) {
        if (a == 0) return {}; // 0^0 tidak terdefinisi
        return {0};
    }

    ll g = find_primitive_root(p);

    // Langkah 1: Selesaikan g^k = b (mod p) ↩
        menggunakan BSGS
    ll k = solve_discrete_log(g, b, p);
    if (k == -1) return {};

    // Langkah 2: Selesaikan a * y = k (mod p - 1)
    ll m = p - 1;
    ll y0, temp_y;
    ll d = extgcd(a, m, y0, temp_y);

    if (k % d != 0) return {}; // Tidak ada solusi jika↩
        d tidak membagi k

    // Basis solusi pertama:
    y0 = ((__int128)y0 * (k / d)) % (m / d);
    if (y0 < 0) y0 += (m / d);
```


New CP Leak
BINUS University
Page 18 of 23

```
    // Langkah 3: Bangkitkan semua d solusi unik
    vector<ll> ans;
    ll step = m / d;
    for (ll i = 0; i < d; ++i) {
        ll y = (y0 + i * step) % m;
        ans.push_back(power(g, y, p));
    }

    sort(ans.begin(), ans.end());
    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    // Contoh Soal 2: 2^x = 13 (mod 59)
    ll a2 = 2, b2 = 13, m2 = 59;
    ll x2 = solve_discrete_log(a2, b2, m2);
    cout << "[Soal 2] " << a2 << "^x = " << b2 << " mod↩
        " << m2 << "\n";
    cout << "x = " << x2 << "\n\n";

    // Contoh Soal 1: x^3 = 8 (mod 11)
    ll a1 = 3, b1 = 8, p1 = 11;
    vector<ll> ans1 = solve_discrete_root(a1, b1, p1);
    cout << "[Soal 1] x^" << a1 << " = " << b1 << " mod↩
        " << p1 << "\n";
    cout << "Solusi x: ";
    for (ll sol : ans1) cout << sol << " ";
    cout << "\n";

    return 0;
}
```

6.24 Matrix Expo

```
template<ll SZ = 2>
struct Mat {
    ll a[SZ][SZ] = {};
    static Mat id() { Mat r; for (ll i = 0; i < SZ; ++i↩
        ) r.a[i][i] = 1; return r; }
    Mat operator*(const Mat& o) const {
        Mat r;
        for (ll i = 0; i < SZ; ++i)
            for (ll k = 0; k < SZ; ++k) if (a[i][k])
                for (ll j = 0; j < SZ; ++j)
                    r.a[i][j] = (r.a[i][j] + (__int128)↩
                        a[i][k] * o.a[k][j]) % MOD;
        return r;
    }
    Mat pow(ll p) const {
        Mat r = id(), b = *this;
        for (; p; p >>= 1, b = b * b) if (p & 1) r = r ↩
            * b;
        return r;
    }
};
```

6.25 Reflection Principle

Grid Path Dasar

$$\text{Paths}\big((0,0)\to(n,m)\big)=\binom{n+m}{n}$$

André's Reflection Principle
Mencari lintasan dari $A(x_1,y_1)$ ke $B(x_2,y_2)$ tanpa menyentuh garis $y=x+k$.
Titik refleksi A terhadap $y=x+k$ adalah A':

$$A'=(y_1-k,\,x_1+k)$$

$$\text{Valid Paths}=\text{Paths}(A\to B)-\text{Paths}(A'\to B)$$

Catalan / Dyck Path ($y <= x$)

$$\text{Dyck Paths}(n)=\binom{2n}{n}-\binom{2n}{n+1}=\frac{1}{n+1}\binom{2n}{n}$$

General Grid $(0,0)->(n,m)$ dengan $y <= x$ dan $n >= m$

$$\text{Valid Paths}=\binom{n+m}{m}-\binom{n+m}{m-1}=\frac{n-m+1}{n+1}\binom{n+m}{m}$$

Bertrand's Ballot Theorem

$$\mathbb{P}(P>Q \text{ throughout})=\frac{p-q}{p+q}\quad(p>q)$$

7 Graph

7.1 Articulation Point and Bridge

```
// Algoritma Tarjan DFS untuk mencari Titik Artikulasi ↩
    (Cut Vertices) dan Jembatan (Bridges) pada graf tak↩
    berarah/multigraf.
// O(V + E).
struct GraphBridges {
    int n, timer;
    vector<vector<pair<int, int>>> gr;
    vector<int> vis, low;
    vector<pair<int, int>> bridges;
    vector<int> articulation_points;

    GraphBridges(int _n) : n(_n), timer(0), gr(_n + 1),↩
        vis(_n + 1, -1), low(_n + 1, -1) {}

    void add_edge(int u, int v, int edge_id) {
        gr[u].push_back({v, edge_id});
        gr[v].push_back({u, edge_id});
    }

    void dfs(int u, int p_edge = -1) {
        vis[u] = low[u] = timer++;
        int kids = 0;
        bool is_art = false;

        for(auto& edge : gr[u]) {
            int v = edge.first, id = edge.second;
            if(id == p_edge) continue;
            if(vis[v] >= 0) {
                low[u] = min(low[u], vis[v]);
            } else {
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if(low[v] > vis[u])
                    bridges.push_back({u, v});
                if(low[v] >= vis[u] && p_edge != -1)
                    is_art = true;
                ++kids;
            }
        }
        if(p_edge == -1 && kids > 1)
            is_art = true;
        if(is_art)
            articulation_points.push_back(u);
    }
};
```

7.2 SCC and Strong Orientation

```
// Algoritma Tarjan untuk mencari SCC pada directed ↩
    graph.
// O(V + E).
const int N = 200005;
vector<int> v[N];
bool vis[N], ins[N];
int disc[N], low[N], gr[N];
stack<int> st;
int id, grid;

void scc(int cur, int par = -1) {
    disc[cur] = low[cur] = ++id;
    vis[cur] = ins[cur] = 1;
    st.push(cur);

    for(int to : v[cur]) {
        if(!vis[to]) {
            scc(to, cur);
            low[cur] = min(low[cur], low[to]);
        } else if(ins[to]) {
            low[cur] = min(low[cur], disc[to]);
        }
    }

    if(low[cur] == disc[cur]) {
        grid++;
        while(true) {
            int u = st.top();
            st.pop();
            gr[u] = grid;
            ins[u] = 0;
            if(u == cur) break;
        }
    }
}
```

7.3 Centroid Decomposition

```
// Centroid Decomposition pada Tree untuk teknik divide↩
    and conquer jalur/lintasan pada pohon.
// Pembentukan dekomposisi O(N log N), kedalaman pohon ↩
    centroid O(log N).
const int N = 200005;
vector<int> adj[N];
int sz[N], par_cen[N];
bool rem[N];

void get_sz(int u, int p = 0) {
    sz[u] = 1;
    for(int v : adj[u]) {
        if(v != p && !rem[v]) {
            get_sz(v, u);
            sz[u] += sz[v];
        }
    }
}

int get_centroid(int u, int p, int total_sz) {
    for(int v : adj[u]) {
        if(v != p && !rem[v] && sz[v] > total_sz / 2)
            return get_centroid(v, u, total_sz);
    }
    return u;
}

int build_cen(int u) {
    get_sz(u, 0);
    int cen = get_centroid(u, 0, sz[u]);
```


New CP Leak
BINUS University
Page 19 of 23

```
    rem[cen] = true;

    for(int v : adj[cen]) {
        if(!rem[v]) {
            int sub_cen = build_cen(v);
            par_cen[sub_cen] = cen;
        }
    }
    return cen;
}
```

7.4 Dinic's Maximum Flow

```
// Algoritma Dinic dengan teknik Capacity Scaling untuk mencari Maximum Network Flow pada graf berarah/tak berarah.
// O(V * E * log(MaxFlow)) dengan scaling, O(E * sqrt(V)) pada bipartite matching.
typedef long long ll;

const ll INF = 1e18;

struct Dinic {
  struct Edge {
    int v;
    ll cap, flow;
    Edge(int _v, ll _cap): v(_v), cap(_cap), flow(0) {}
  };

  int n;
  ll lim;
  vector<vector<int>> gr;
  vector<Edge> e;
  vector<int> idx, lv;

  bool has_path(int s, int t) {
    queue<int> q;
    q.push(s);
    lv.assign(n, -1);
    lv[s] = 0;
    while(!q.empty()) {
      int c = q.front();
      q.pop();
      if(c == t)
        break;
      for(auto& i : gr[c]) {
        ll cur_flow = e[i].cap - e[i].flow;
        if(lv[e[i].v] == -1 && cur_flow >= lim) {
          lv[e[i].v] = lv[c] + 1;
          q.push(e[i].v);
        }
      }
    }
    return lv[t] != -1;
  }

  ll get_flow(int s, int t, ll left) {
    if(!left || s == t)
      return left;
    while(idx[s] < (int) gr[s].size()) {
      int i = gr[s][idx[s]];
      if(lv[e[i].v] == lv[s] + 1) {
        ll add = get_flow(e[i].v, t, min(left, e[i].cap - e[i].flow));
        if(add) {
          e[i].flow += add;
          e[i ^ 1].flow -= add;
          return add;
        }
      }
      ++idx[s];
    }
    return 0;
  }

  Dinic(int vertices, bool scaling = 1) :
    n(vertices), lim(scaling ? (1LL << 60) : 1), gr(n) {}

  void add_edge(int from, int to, ll cap, bool directed = 1) {
    gr[from].push_back(e.size());
    e.emplace_back(to, cap);
    gr[to].push_back(e.size());
    e.emplace_back(from, directed ? 0 : cap);
  }

  ll get_max_flow(int s, int t) {
    ll res = 0;
    while(lim) {
      while(has_path(s, t)) {
        idx.assign(n, 0);
        while(ll add = get_flow(s, t, INF))
          res += add;
      }
      lim >>= 1;
    }
    return res;
  }
};
```

7.5 Minimum Cost Maximum Flow

```
// MCMF menggunakan Shortest Path Faster Algorithm (SPFA) untuk mencari alur maksimum berbiaya minimum.
// O(F * V * E) dengan F adalah kapasitas alur maksimum.
typedef long long ll;

using FlowT = ll;
using CostT = ll;

const FlowT F_INF = 1e18;
const CostT C_INF = 1e18;
const int MAX_V = 1e5 + 5;
const int MAX_E = 1e6 + 5;

namespace MCMF {
  int n, E;
  int adj[MAX_E], nxt[MAX_E], lst[MAX_V], frm[MAX_V], vis[MAX_V];
  FlowT cap[MAX_E], flw[MAX_E], totalFlow;
  CostT cst[MAX_E], dst[MAX_V], totalCost;

  void init(int _n) {
    n = _n;
    fill_n(lst, n, -1), E = 0;
  }
  void add(int u, int v, FlowT ca, CostT co) {
    adj[E] = v, cap[E] = ca, flw[E] = 0, cst[E] = +co;
    nxt[E] = lst[u], lst[u] = E++;
    adj[E] = u, cap[E] =  0, flw[E] = 0, cst[E] = -co;
    nxt[E] = lst[v], lst[v] = E++;
  }
  int spfa(int s, int t) {
    fill_n(dst, n, C_INF);
    fill_n(vis, n, 0);
    dst[s] = 0;
    vis[s] = 1;
    queue<int> que;
    que.push(s);
    while(que.size()) {
      int u = que.front();
      que.pop();
      vis[u] = 0;
      for(int e = lst[u]; e != -1; e = nxt[e])
        if(flw[e] < cap[e]) {
          int v = adj[e];
          if(dst[v] > dst[u] + cst[e]) {
            dst[v] = dst[u] + cst[e];
            frm[v] = e;
            if(!vis[v]) {
              vis[v] = 1;
              que.push(v);
            }
          }
        }
    }
    return dst[t] < C_INF;
  }
  pair<FlowT, CostT> solve(int s, int t) {
    totalCost = 0, totalFlow = 0;
    while(1) {
      if(!spfa(s, t))
        break;
      FlowT mn = F_INF;
      for(int v = t, e = frm[v]; v != s; v = adj[e ^ 1], e = frm[v])
        mn = min(mn, cap[e] - flw[e]);
      for(int v = t, e = frm[v]; v != s; v = adj[e ^ 1], e = frm[v]) {
        flw[e] += mn;
        flw[e ^ 1] -= mn;
      }
      totalFlow += mn;
      totalCost += mn * dst[t];
    }
    return {totalFlow, totalCost};
  }
};
```

7.6 Flows with Demands

Sirkulasi Alur Jaringan dengan Batas Bawah dan Batas Atas Kapasitas (Flows with Demands) via Supersource/Supersink. NOTE: S_1 = supersource baru, T_1 = supersink baru. T_0, S_0 = source/sink asli. Pastikan semua edge (S_1, v) dan (v, T_1) terisi penuh ($flow == cap$) agar feasible.

```
let S0 be the source and T0 be the original sink
1. add 2 additional nodes, call them S1 and T1
2. connect S0 to nodes normally
3. connect nodes to T0 normally
4. for each edge(U, V), cap = original cap - demand
5. for each node N :
   1. add an edge(S1, N), cap = sum of inward demand to N
   2. add an edge(N, T1), cap = sum of outward demand from N
6. add an edge(T0, S0), cap = INF
7. the above is not a typo!
8. run max flow normally
9. for each edge(S1, V) and (U, T1), check if flow == cap
if step #9 fails, then it is not possible to satisfy the given demand
```

Mathematically, let $d(e)$ be the demand of edge e. Let V be the set of every vertex in the graph.

New CP Leak
BINUS University
Page 20 of 23

- $c'(S_1, v) = \sum_{u \in V} d(u, v)$ for each edge (S_1, v).
- $c'(v, T_1) = \sum_{w \in V} d(v, w)$ for each edge (v, T_1).
- $c'(u, v) = c(u, v) - d(u, v)$ for each edge (u, v) in the old network.
- $c'(T_0, S_0) = \infty$

7.7 Hungarian

```
// Algoritma Hungarian (Kuhn-Munkres) untuk Maximum Weight Bipartite Matching pada graf bipartit berbobot.
// O(N^3).
template <typename TD> struct Hungarian {
    TD INF = 1e9;
    int n;
    vector<vector<TD>> adj;
    vector<TD> hl, hr, slk;
    vector<int> fl, fr, vl, vr, pre;
    deque<int> q;

    Hungarian(int _n) : n(_n), adj(_n, vector<TD>(_n, 0)) {}

    int check(int i) {
        if(vl[i] = 1, fl[i] != -1)
            return q.push_back(fl[i]), vr[fl[i]] = 1;
        while(i != -1)
            swap(i, fr[fl[i] = pre[i]]);
        return 0;
    }

    void bfs(int s) {
        slk.assign(n, INF);
        vl.assign(n, 0);
        vr = vl;
        q.assign(vr[s] = 1, s);
        for(TD d;;) {
            for(; !q.empty(); q.pop_front()) {
                for(int i = 0, j = q.front(); i < n; i++) {
                    if(d = hl[i] + hr[j] - adj[i][j], !vl[i] && d <= slk[i]) {
                        if(pre[i] = j, d)
                            slk[i] = d;
                        else if(!check(i))
                            return;
                    }
                }
            }
            d = INF;
            for(int i = 0; i < n; i++) if(!vl[i] && d > slk[i])
                d = slk[i];
            for(int i = 0; i < n; i++) {
                if(vl[i])
                    hl[i] += d;
                else
                    slk[i] -= d;
                if(vr[i])
                    hr[i] -= d;
            }
            for(int i = 0; i < n; i++) if(!vl[i] && !slk[i] && !check(i))
                return;
        }
    }

    TD solve() {
        fl.assign(n, -1);
        fr = fl;
        hl.assign(n, 0);
        hr = hl;
        pre.assign(n, 0);
        for(int i = 0; i < n; i++)
            hl[i] = *max_element(adj[i].begin(), adj[i].begin() + n);
        for(int i = 0; i < n; i++)
            bfs(i);
        TD ret = 0;
        for(int i = 0; i < n; i++) {
            if(fl[i] != -1)
                ret += adj[i][fl[i]];
        }
        return ret;
    }
};
```

7.8 Edmond's Blossom

```
// Algoritma Edmond's Blossom untuk mencari Maximum Cardinality Matching pada graf umum (non-bipartit) 1-based index.
// O(V^2 * E).
struct Blossom {
  vector<int> vis, dad, orig, match, aux;
  vector<vector<int>> conn;
  int t, N;
  queue<int> Q;

  void augment(int u, int v) {
    int pv = v;
    do {
      pv = dad[v];
      int nv = match[pv];
      match[v] = pv;
      match[pv] = v;
      v = nv;
    } while(u != pv);
  }

  int lca(int v, int w) {
    ++t;
    while(true) {
      if(v) {
        if(aux[v] == t)
          return v;
        aux[v] = t;
        v = orig[dad[match[v]]];
      }
      swap(v, w);
    }
  }

  void blossom(int v, int w, int a) {
    while(orig[v] != a) {
      dad[v] = w;
      w = match[v];
      if(vis[w] == 1) {
        Q.push(w);
        vis[w] = 0;
      }
      orig[v] = orig[w] = a;
      v = dad[w];
    }
  }

  bool bfs(int u) {
    fill(vis.begin(), vis.end(), -1);
    iota(orig.begin(), orig.end(), 0);
    Q = queue<int>();
    Q.push(u);
    vis[u] = 0;
    while(!Q.empty()) {
      int v = Q.front();
      Q.pop();
      for(int x : conn[v]) {
        if(vis[x] == -1) {
          dad[x] = v;
          vis[x] = 1;
          if(!match[x]) {
            augment(u, x);
            return 1;
          }
          Q.push(match[x]);
          vis[match[x]] = 0;
        } else if(vis[x] == 0 && orig[v] != orig[x]) {
          int a = lca(orig[v], orig[x]);
          blossom(x, v, a);
          blossom(v, x, a);
        }
      }
    }
    return false;
  }

  Blossom(int n) :
    vis(n + 1), dad(n + 1), orig(n + 1), match(n + 1),
    aux(n + 1), conn(n + 1), t(0), N(n) {
    for(int i = 0; i <= n; ++i) {
      conn[i].clear();
      match[i] = aux[i] = dad[i] = 0;
    }
  }

  void add_edge(int u, int v) {
    conn[u].push_back(v);
    conn[v].push_back(u);
  }

  int solve() {
    int ans = 0;
    vector<int> V(N);
    iota(V.begin(), V.end(), 1);
    shuffle(V.begin(), V.end(), mt19937(0x94949));
    for(auto x : V) {
      if(!match[x]) {
        for(auto y : conn[x]) {
          if(!match[y]) {
            match[x] = y, match[y] = x;
            ++ans;
            break;
          }
        }
      }
    }
    for(int i = 1; i <= N; ++i) {
      if(!match[i] && bfs(i))
        ++ans;
    }
    return ans;
  }
};
```

7.9 Eulerian Path or Cycle

New CP Leak
BINUS University
Page 21 of 23

```
// Mencari lintasan/siklus Eulerian pada graf tak berarah sederhana dengan mengunjungi setiap sisi tepat satu kali.
// O(E log V) menggunakan set.
vector<set<int>> g;
vector<int> ans;

void dfs(int u) {
  while(g[u].size()) {
    int v = *g[u].begin();
    g[u].erase(v);
    g[v].erase(u);
    dfs(v);
  }
  ans.push_back(u);
}
```

7.10 Hierholzer's Algorithm

```
// Algoritma Hierholzer untuk mencari lintasan atau siklus Eulerian pada graf berarah.
// O(V + E).
vector<int> hierholzer(int start_node, vector<vector<int>>& adj) {
    stack<int> path;
    vector<int> euler;
    path.push(start_node);
    int cur = start_node;

    while(!path.empty()) {
        if(!adj[cur].empty()) {
            path.push(cur);
            int nxt = adj[cur].back();
            adj[cur].pop_back();
            cur = nxt;
        } else {
            euler.push_back(cur);
            cur = path.top();
            path.pop();
        }
    }
    reverse(euler.begin(), euler.end());
    return euler;
}
```

7.11 2-SAT

```
// 2-SAT Solver menggunakan algoritma Kosaraju untuk mencari SSC pada graf implikasi boolean.
// O(V + E) dengan V jumlah variabel dan E jumlah klausul.
struct TwoSAT {
  int n;
  vector<vector<int>> g, gr;
  vector<int> comp, topological_order, answer;
  vector<bool> vis;

  TwoSAT() {}
  TwoSAT(int _n) :
    n(_n), g(2 * n), gr(2 * n), comp(2 * n), answer(2 * n), vis(2 * n) {}

  void add_edge(int u, int v) {
    g[u].push_back(v);
    gr[v].push_back(u);
  }

  void add_clause_or(int i, bool f, int j, bool p) {
    add_edge(i + (f ? n : 0), j + (p ? 0 : n));
  }

  void add_clause_xor(int i, bool f, int j, bool p) {
    add_clause_or(i, f, j, p);
    add_clause_or(i, !f, j, !p);
  }

  void add_clause_and(int i, bool f, int j, bool p) {
    add_clause_xor(i, !f, j, p);
  }

  void dfs(int u) {
    vis[u] = true;
    for(const auto& v : g[u])
      if(!vis[v])
        dfs(v);
    topological_order.push_back(u);
  }

  void scc(int u, int id) {
    vis[u] = true;
    comp[u] = id;
    for(const auto& v : gr[u])
      if(!vis[v])
        scc(v, id);
  }

  bool satisfiable() {
    topological_order.clear();
    fill(vis.begin(), vis.end(), false);
    for(int i = 0; i < 2 * n; i++)
      if(!vis[i])
        dfs(i);
    fill(vis.begin(), vis.end(), false);
    reverse(topological_order.begin(), topological_order.end());
    int id = 0;
    for(const auto& v : topological_order)
      if(!vis[v])
        scc(v, id++);
    for(int i = 0; i < n; i++) {
      if(comp[i] == comp[i + n])
        return false;
      answer[i] = (comp[i] > comp[i + n] ? 1 : 0);
    }
    return true;
  }
};
```

7.12 Edge Coloring

```
// Pewarnaan sisi greedy (Edge Coloring) pada pohon/hutan sehingga sisi yang bertetangga memiliki warna berbeda.
// O(V + E).
void colorEdges(int ptr, vector<vector<pair<int, int>>>& gra, vector<int>& edgeColors, vector<bool>& isVisited) {
  queue<int> q;
  int c = 0;
  unordered_set<int> colored;
  if (isVisited[ptr]) return;

  isVisited[ptr] = 1;
  for (size_t i = 0; i < gra[ptr].size(); i++) {
    if (edgeColors[gra[ptr][i].second] != -1)
      colored.insert(edgeColors[gra[ptr][i].second]);
  }
  for (size_t i = 0; i < gra[ptr].size(); i++) {
    if (!isVisited[gra[ptr][i].first])
      q.push(gra[ptr][i].first);
    if (edgeColors[gra[ptr][i].second] == -1) {
      while (colored.find(c) != colored.end())
        c++;
      edgeColors[gra[ptr][i].second] = c;
      colored.insert(c);
      c++;
    }
  }
  while (!q.empty()) {
      int temp = q.front();
      q.pop();
      colorEdges(temp, gra, edgeColors, isVisited);
  }
}
```

7.13 K-th Shortest Path

```
// Menemukan panjang lintasan terpendek ke-k dari simpul s ke t pada graf berbobot positif menggunakan Priority Queue (Dijkstra-based).
// O(K * E log(K * V)).
const int MAXN = 1005;
vector<pair<int, int>> g[MAXN];
int vis_cnt[MAXN];

int kth_shortest_path(int s, int t, int k, int n) {
    fill(vis_cnt, vis_cnt + n + 1, 0);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, s});

    while(!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if(vis_cnt[u] >= k) continue;
        vis_cnt[u]++;

        if(u == t && vis_cnt[t] == k)
            return d;

        for(auto& edge : g[u]) {
            int v = edge.first, w = edge.second;
            if(vis_cnt[v] < k)
                pq.push({d + w, v});
        }
    }
    return -1;
}
```

8 OEIS

8.1 A000108 (Catalan Number)

```
// Algoritma/Fungsi: Menghitung Bilangan Catalan C(n) = C(2n, n) / (n+1) dan Bilangan Super Catalan (Schroder).
// Kompleksitas Waktu: Catalan DP O(N^2), Super Catalan O(N).
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll findCatalan(int n) {
    if(n <= 1) return 1;
```


New CP Leak
BINUS University
Page 22 of 23

```
    vector<ll> catalan(n + 1, 0);
    catalan[0] = catalan[1] = 1;
    for(int i = 2; i <= n; i++) {
        for(int j = 0; j < i; j++)
            catalan[i] += catalan[j] * catalan[i - j - ↩
                1];
    }
    return catalan[n];
}

ll superCat(int n) {
    if(n <= 1) return 1;
    vector<ll> s(n + 1, 0);
    s[0] = s[1] = 1;
    for(int i = 2; i <= n; ++i) {
        s[i] = (3 * (2 * i - 1) * s[i - 1] - (i - 2) * ↩
            s[i - 2]) / (i + 1);
    }
    return s[n];
}
```

8.2 A000110 (Bell Numbers)

Total number of partitions of n distinct elements.
$B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$
For p prime,
$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$

8.3 A000127

Maximal number of regions obtained by joining n points around a circle by straight lines.
$f(n) = (n^4 - 6n^3 + 23n^2 - 18n + 24)/24$
$1, 2, 4, 8, 16, 31, 57, 99, 163, 256, 386, 562, 794, 1093, 1471, 1941,$
$2517, 3214, 4048, 5036, 6196, 7547, 9109, 10903, 12951,$
$15276, 17902, 20854, 24158, 27841, 31931, 36457, 41449, 46938,$
$52956, 59536, 66712, 74519, 82993, 92171, 102091, 112792, 124314$

8.4 A001434

Number of graphs with n nodes and n edges
$0, 0, 1, 2, 6, 21, 65, 221, 771, 2769, 10250, 39243, 154658,$
$628635, 2632420, 11353457, 50411413, 230341716, 1082481189,$
$5228952960, 25945377057, 132140242356, 690238318754$

8.5 A018819

Binary partition function: number of partitions of n into powers of 2
$f(2m+1) = f(2m);\ f(2m) = f(2m-1) + f(m)$
$1, 1, 2, 2, 4, 4, 6, 6, 10, 10, 14, 14, 20, 20, 26, 26, 36, 36,$
$46, 46, 60, 60, 74, 74, 94, 94, 114, 114, 140, 140, 166, 166,$
$202, 202, 238, 238, 284, 284, 330, 330, 390, 390, 450, 450,$
$524, 524, 598, 598, 692, 692, 786, 786, 900, 900, 1014, 1014,$
$1154, 1154, 1294, 1294$

8.6 A092098

```
// Algoritma/Fungsi: 3-Portolan Numbers untuk ↩
    menghitung jumlah daerah yang terbentuk dari n-↩
    seksi sudut pada segitiga sama sisi.
// Kompleksitas Waktu: O(N^2).
typedef long long ll;
ll solve(ll n){
    ll res = (n % 2 == 1 ? 3 * n * n - 3 * n + 1 : 3 * ↩
        n * n - 6 * n + 6);
    const int bats = n / 2 - 1;
    for(ll i = 1; i <= bats; i++){
        for(ll j = 1; j <= bats; j++){
            ll num = i * (n - j) * n;
            ll denum = (n - i) * j + i * (n - j);
            res -= 6 * (num % denum == 0 && num / denum↩
                <= bats);
        }
    }
    return res;
}
```


New CP Leak
BINUS University

Page 23 of 23